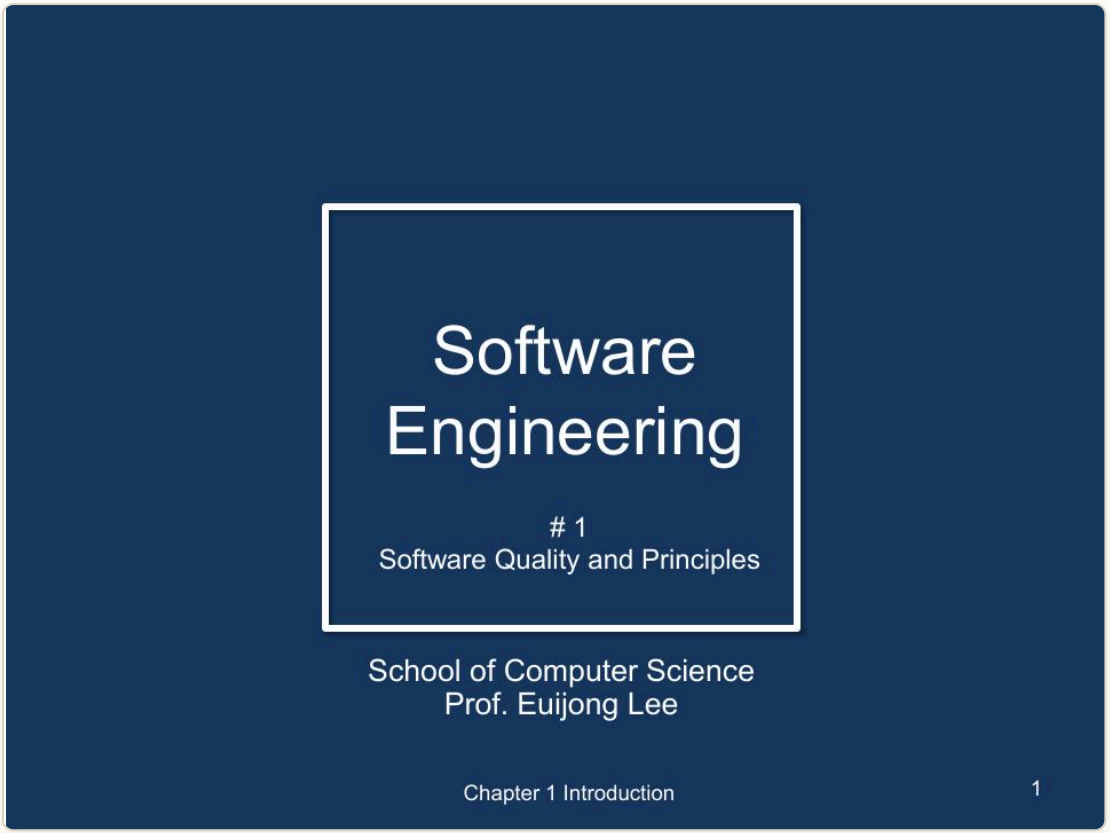


소프트웨어공학 #1 — 소프트웨어 품질과 원칙

- Chapter 1 Introduction · 이의종 교수님

✍️ 메모

💡 이해 1장 큰 줄기: SW란 → SE란 → SW 개발의 특징 → SE 세부(좋은 SW 속성) → SW 다양성 → 윤리

Chapter1. Introduction : Software Quality and Principles

Advise from the creator of C++





https://www.youtube.com/watch?v=-Qxl-RP6-HM
https://www.stroustrup.com/index.html

C++ 창시자의 조언

- Bjarne Stroustrup 인터뷰 영상 소개

✍️ 메모

— 넘어가도 되는 쪽

Chapter1. Introduction : Software Quality and Principles

Advise from the creator of C++



- 본인의 커리어를 지나치게 전문화 하지 마세요. (don't be overspecialized)
- 미래를 알고 있다고 너무 확신하지 마세요.
- 융통성을 가지고 (Be flexible), 커리어와 직업은 장기적인 것이라는 것을 기억하세요.
- 너무 많은 젊은이들이 자신이 뭔가를 최적화 할 수 있다고 생각하고, Right thing이 아닌 것에 전문화 하면서 많은 시간을 소비했다는 것을 깨닫습니다. 그리고 그 과정에서 번-아웃 됩니다.
- **우정을 쌓고 컴퓨팅 밖에서의 삶을 사는데 충분한 시간을 보내지 않았기 때문이에요.**
- 나는 많은 종류의 사람을 만납니다. 뭐라고 부르는지 모르겠는데, " Junior Geeks " ?
- 그들은 **오직 프로그래밍, AI, 그래픽 등의 컴퓨터에 대한 전문성만이 중요하다고 생각**합니다.
- 그렇지 않아요. 만약 그들이 다른 일을 하지 않는다면...
- 아이디어를 소통하지 않는다면, (혼자 하는) 스도쿠를 할 수 있습니다.
- **소통(Communication) 을 해야 해요.**
- 많은 **괴짜(Nerd)들이 이 사실을 잊고 있습니다**.
- 그들은 최고의 코드를 작성하기만 하면 세상을 바꿀 수 있다고 생각합니다.
- 하지만 경청할 줄 알아야 합니다.
- **사용자와 소통하고 그들로 부터 배울 수 있어야 합니다.**
- 그리고 **자신의 아이디어를 사용자에게 전달할 수 있어야 합니다.**

C++ 창시자의 조언 ①

- (슬라이드가 이미 한국어) 커리어를 지나치게 전문화하지 말 것, 미래를 너무 확신하지 말 것, 융통성
- 컴퓨팅 밖의 삶과 우정, **소통(Communication)**이 중요
- 경청하고, 사용자에게서 배우고, 내 아이디어를 전달할 줄 알아야

✍️ 메모

💡 이해 동기 부여 슬라이드 — 시험 비중 낮음

Chapter1. Introduction : Software Quality and Principles

Advise from the creator of C++



- 그러니 코딩만 할 수는 없습니다.
- **문화와 아이디어를 표현하는 방법에 대해 뭔가를 해야 합니다.**
- 내 말은, 나는 역사와 수학에 보낸 시간을 결코 후회하지 않는다는 것입니다.
- 수학을 정신을 날카롭게 하고, 역사는 자신의 한계에 대한 아이디어와 세상에 무슨 일이 일어나고 있는지를 알려줍니다.
- 그러니 너무 확신하지 마세요. **균형 잡힌 삶을 살기 위한 시간**을 가져보세요.
- **그리고 기회를 잡을 준비를 하세요.**
- 내 말은, 폭넓은 교육, 폭넓은 기술 세트(교육을 통해 기본적으로 기술 포트폴리오를 구축하는 것)는 기회가 왔을 때 이를 활용할 수 있다는 것을 의미합니다.
- 가끔 기회를 알아차릴 수 있습니다. 우리에게는 많은 기회가 있습니다.
- 하지만, 그 중 상당수는 우리가 활용하지 못하거나 알아차리지 못합니다.
- **저는 표준 컴퓨터 과학, 컴파일러, 여러 언어를 공부하는 등 상당히 폭넓은 교육**을 받았습니다.
- 당시 제가 알고 있었던 언어는 약 20여가지 였던 것 같아요. 그리고 머신 아키텍처와 운영체제도 다뤄봤죠.
- **이 기술 세트는 유용했어요(skill set turned out to be useful)**

C++ 창시자의 조언 ②

- 코딩만 할 수는 없다 — 역사·수학, 균형 잡힌 삶
- 폭넓은 교육과 기술 세트가 기회를 잡게 해 준다

✍️ 메모

— 넘어가도 되는 쪽

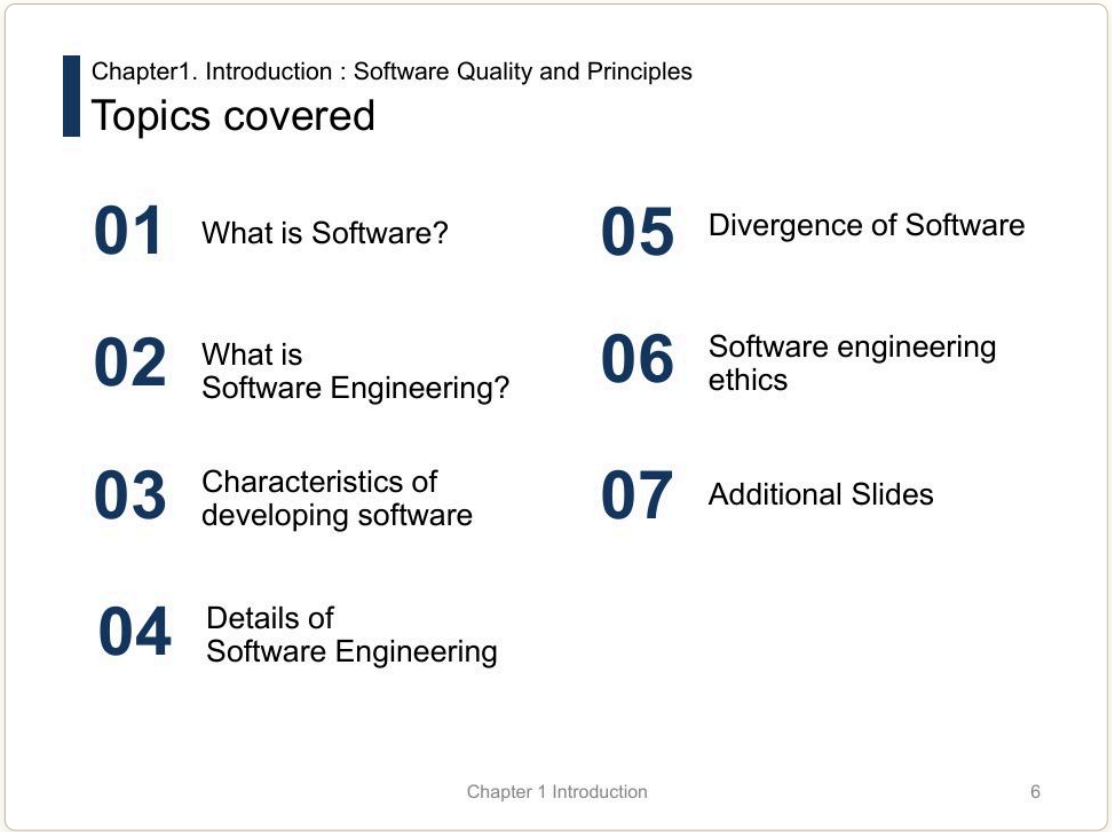


다루는 내용

- **Professional software development** — 소프트웨어 공학이 뜻하는 것
- **Software engineering ethics** — SE에 영향을 주는 윤리 문제 소개

✍️ 메모

— 넘어가도 되는 쪽

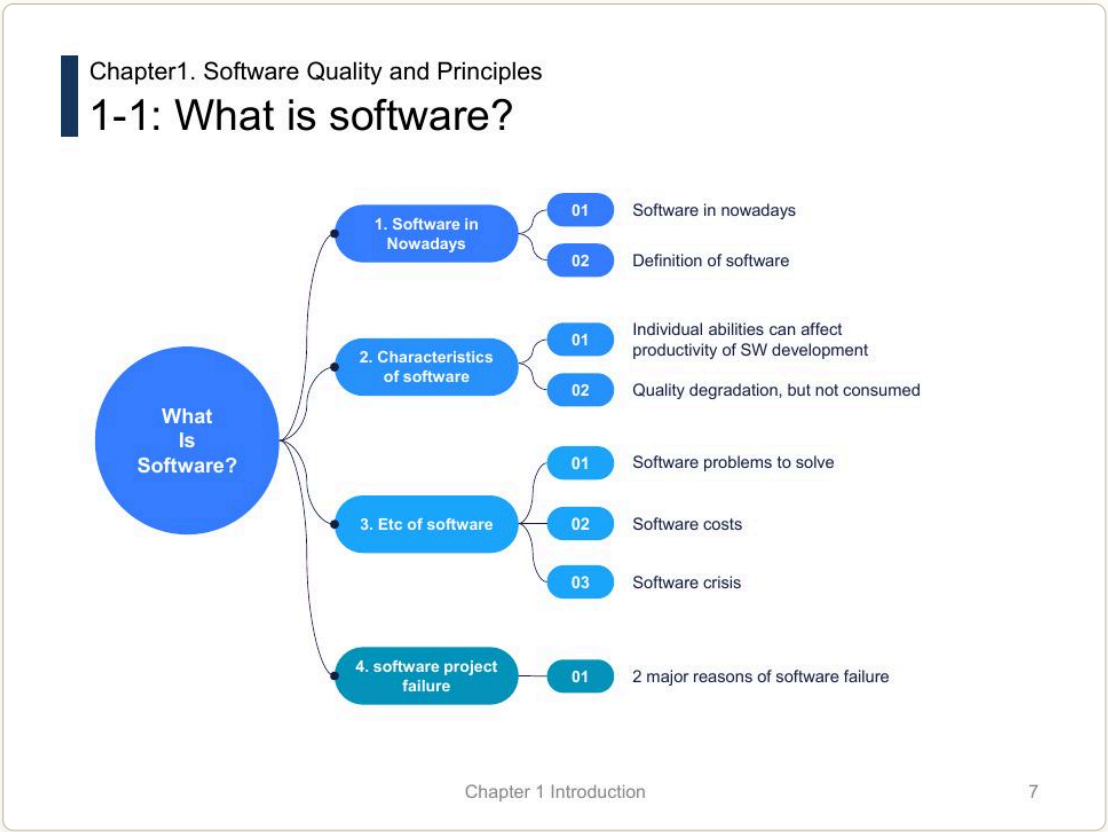


다루는 내용 (목차)

- 01 SW란? / 02 SE란? / 03 SW 개발의 특징 / 04 SE 세부 / 05 SW의 다양성 / 06 SE 윤리 / 07 추가 슬라이드

✍️ 메모

— 넘어가도 되는 쪽



1-1: 소프트웨어란?

- 오늘날의 SW — 정의
- SW의 특성 — 개인 능력이 생산성에 영향 / 품질은 떨어지지만 소모되지 않음
- 기타 — 풀어야 할 문제, SW 비용, SW 위기
- SW 프로젝트 실패의 두 가지 주요 원인

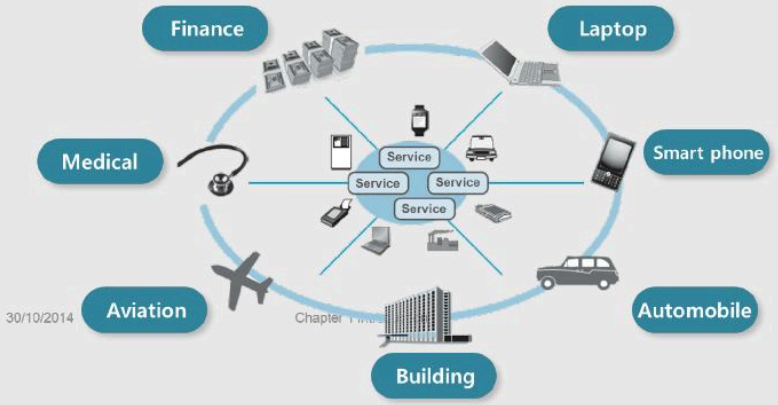
✍️ 메모

💡 이해 절 표지 — 아래 쪽들의 지도

Software in Nowadays

Software in nowadays

[Nowadays, software is used in several fields of society]



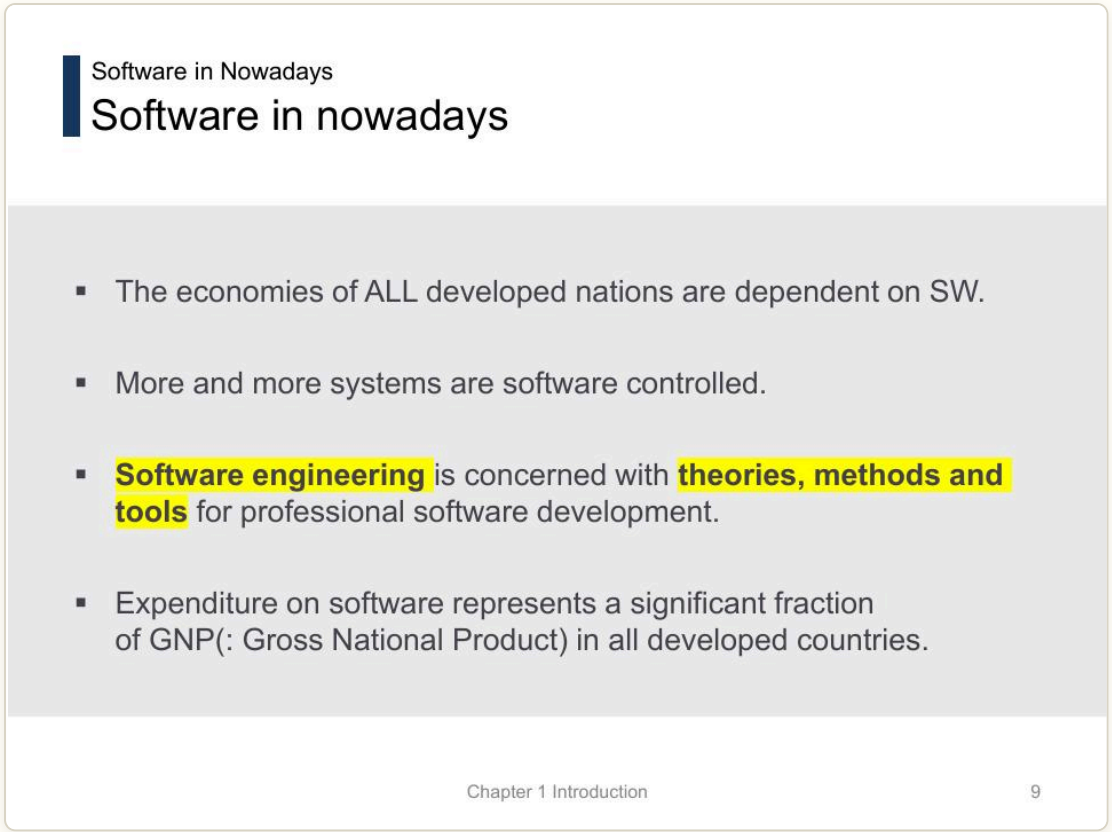
30/10/2014 Chapter 1 Introduction 8

오늘날의 소프트웨어

- 오늘날 SW는 사회 여러 분야에서 쓰인다 (그림)

✍ 메모

— 넘어가도 되는 쪽

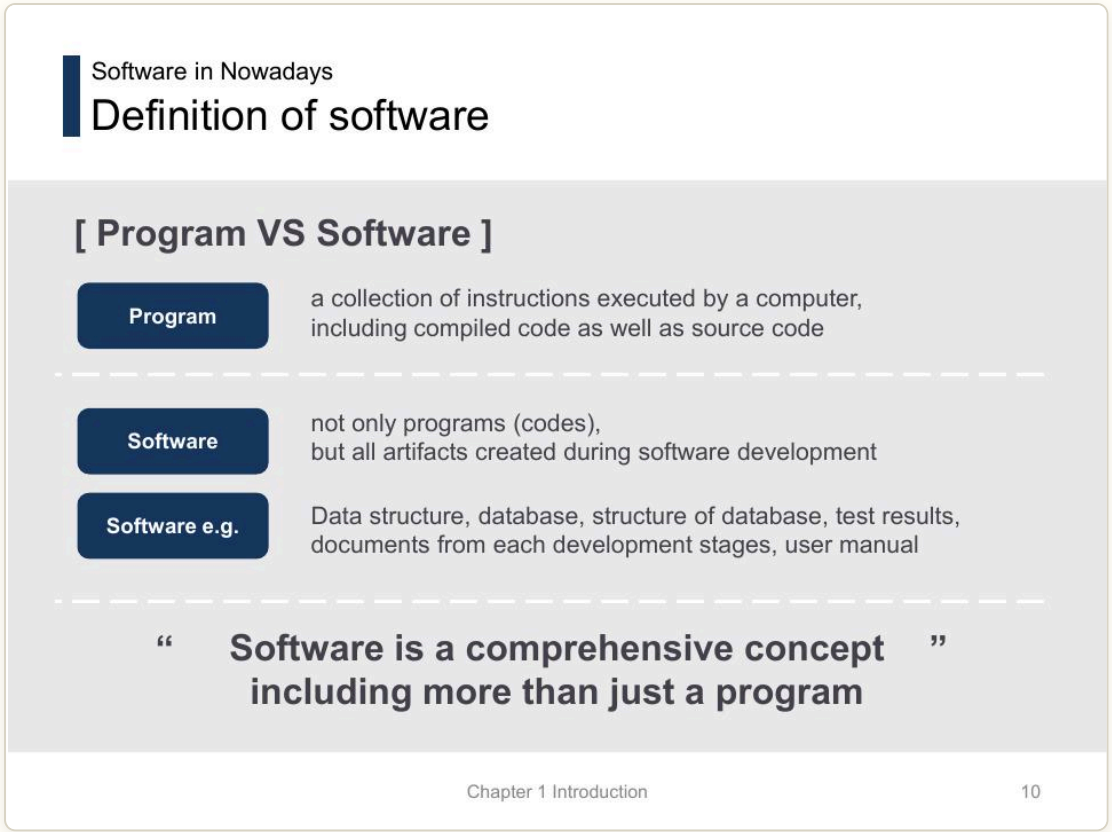


오늘날의 소프트웨어 (계속)

- 모든 선진국의 경제가 SW에 **의존**
- 점점 더 많은 시스템이 **SW로 제어**됨
- SE는 전문적 SW 개발을 위한 **이론·방법·도구**를 다룬다
- SW 지출은 모든 선진국 GNP의 상당 부분

✍️ 메모

★ 시험 **SE = theories, methods and tools for professional software development**

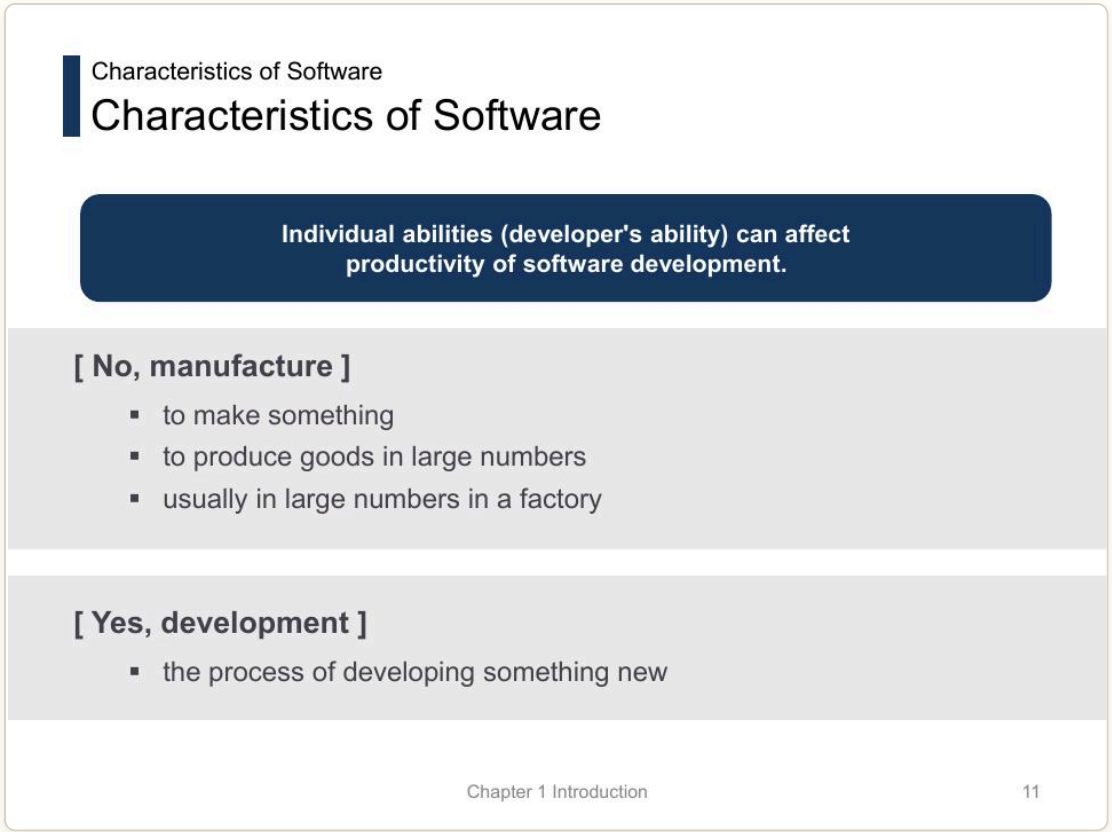


소프트웨어의 정의 — 프로그램 vs 소프트웨어

- **Program**: 컴퓨터가 실행하는 명령어 모음 (컴파일된 코드와 소스 코드)
- **Software**: 프로그램(코드)만이 아니라 **개발 중 만들어진 모든 산출물**
  - 예: 자료 구조, DB와 그 구조, 테스트 결과, 단계별 문서, 사용자 매뉴얼
- SW는 프로그램보다 **넓은 개념**

✍ 메모

★ 시험 **Software ⊃ Program — '문서·테스트 결과도 SW' O/X 단골**



SW의 특성 ① 개인 능력이 생산성에 영향

- 개발자의 **개인 능력**이 SW 개발 생산성에 큰 영향
  - SW는 **제조(manufacture)가 아니다** — 공장에서 같은 것을 대량 생산하는 게 아님
  - SW는 **개발(development)** — 새로운 것을 만들어 내는 과정

✍ 메모

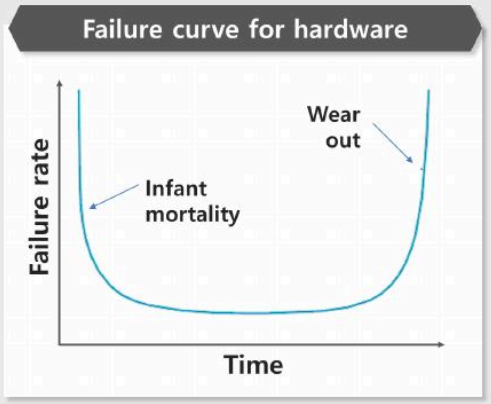
★ 시험 **SW is developed, not manufactured**

Characteristics of Software

Characteristics of Software

[Quality degradation, but not consumed]

- Hardware has a high initial failure rate but can be used for a long time after error correction
- Hardware failure is increased rapidly because of environmental condition (e.g., dust, vibration, and heat)



Chapter 1 Introduction 12

SW의 특성 ② 품질은 떨어지지만 소모되지 않는다 (HW)

- 하드웨어는 **초기 고장률이 높지만** 고친 뒤에는 오래 쓴다
- 하드웨어 고장은 먼지·진동·열 같은 **환경 때문에** 빠르게 늘어난다
- 그림: HW 고장 곡선 — 욕조(bathtub) 모양: 초기 고장(infant mortality) → 안정 → 마모(wear out)

✍ 메모

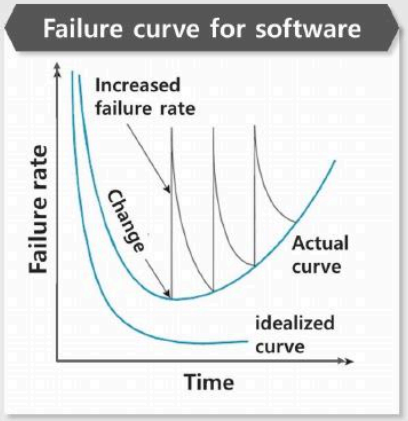
★ 시험 HW 곡선 = 욕조 모양 (초기 높음 → 낮음 → 마모로 다시 높음)

Characteristics of Software

Characteristics of Software

[Quality degradation, but not consumed (2)]

- Unlike hardware, software has no damage by parts (i.e., Environments have little impact on the software)
- In the idealized software curve, the failure rate is decreased after error corrections
- Software can reflect requirements in stage of use (i.e., software is updated to reflect user's requirement)
- The changes may cause increasing of failure rate



Chapter 1 Introduction 13

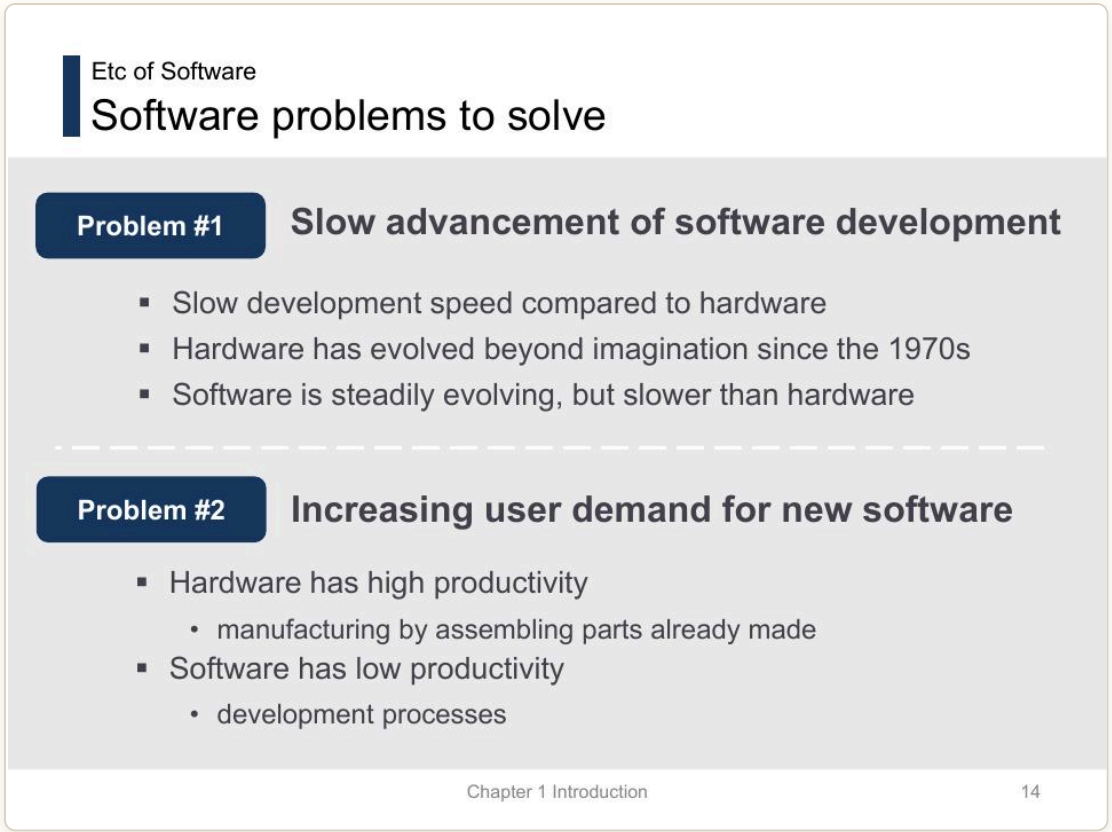
SW의 특성 ② (계속) — SW 고장 곡선

- SW는 하드웨어와 달리 **부품 손상이 없다** (환경 영향 적음)
- **이상적 SW 곡선**: 오류를 고치면 고장률이 줄어든다
- SW는 사용 중에도 요구사항을 반영해 **갱신**된다
  - 그 **변경 때문에 고장률이 다시 오를 수** 있다
- 그림: 실제 곡선(actual) — 변경할 때마다 고장률이 튀어 오르며 점점 올라감

✍ 메모

★ 시험 SW는 닳지 않는다(not wear out) — 대신 '변경'으로 품질이 떨어진다 (deteriorate)

⚠ 주의 SW 고장의 원인 = 환경이 아니라 변경

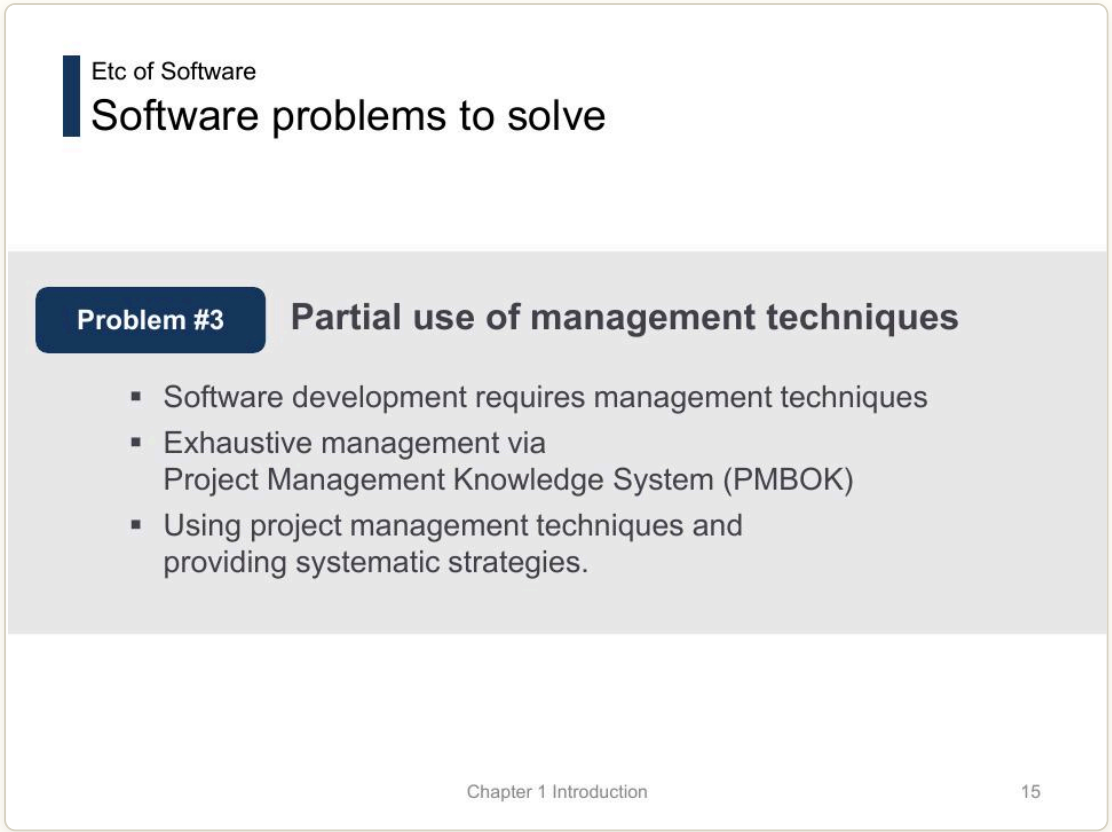


풀어야 할 SW 문제 ①②

- **문제 1: 느린 SW 발전 속도**
  - 하드웨어보다 개발 속도가 느림 — HW는 1970년대 이후 상상 이상으로 발전, SW는 꾸준하지만 더 느림
- **문제 2: 새 SW에 대한 사용자 요구 증가**
  - HW는 이미 만든 부품을 조립하니 생산성이 높다
  - SW는 개발 과정이라 생산성이 낮다

✍️ 메모

— 넘어가도 되는 쪽

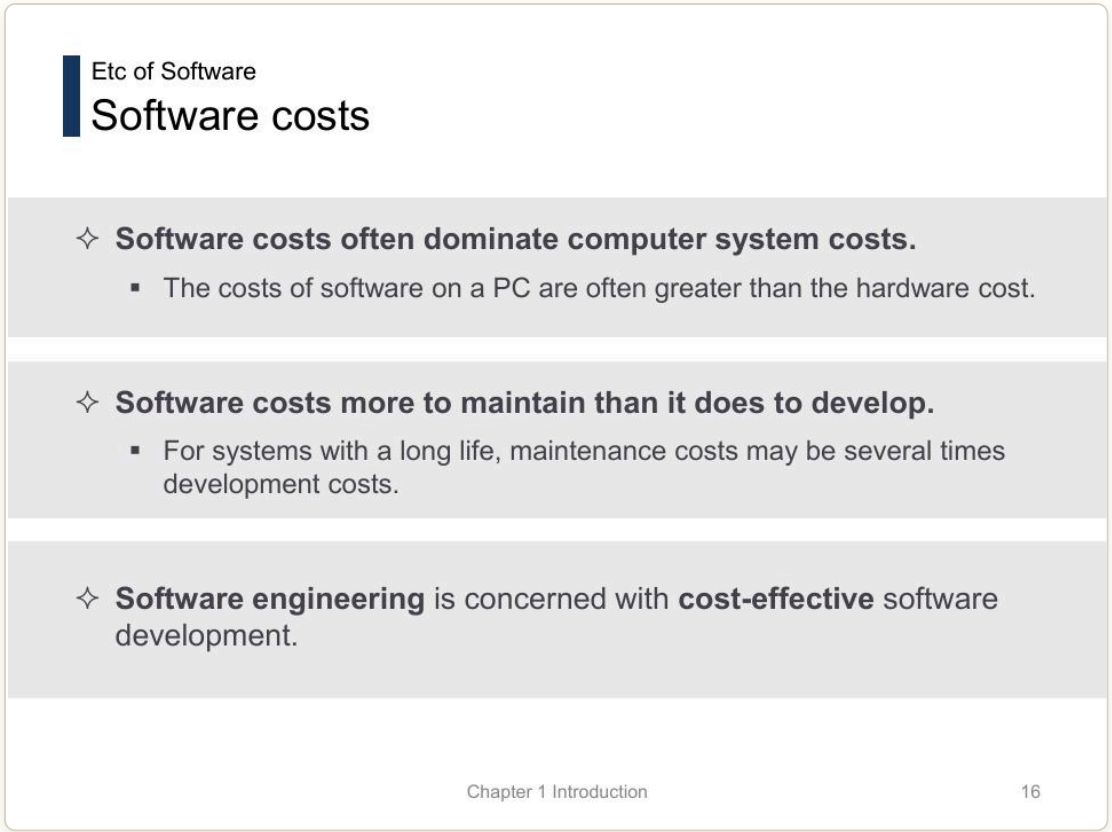


풀어야 할 SW 문제 ③

- **문제 3: 관리 기법을 부분적으로만 사용**
  - SW 개발에는 관리 기법이 필요
  - **PMBOK**(프로젝트 관리 지식 체계)로 철저히 관리
  - 프로젝트 관리 기법과 체계적 전략 제공

✍ 메모

★ 시험 문제 3개: 느린 발전 · 늘어나는 요구 · 관리 기법 부분 사용



SW 비용

- SW 비용이 컴퓨터 시스템 비용의 **대부분**인 경우가 많다
  - PC에서도 SW 비용이 HW보다 큰 경우가 많다
- SW는 **개발보다 유지보수에 더 많은 비용**
  - 수명이 긴 시스템은 유지보수 비용이 개발 비용의 몇 배
- SE는 **비용 효율적인** SW 개발을 다룬다

✍️ 메모

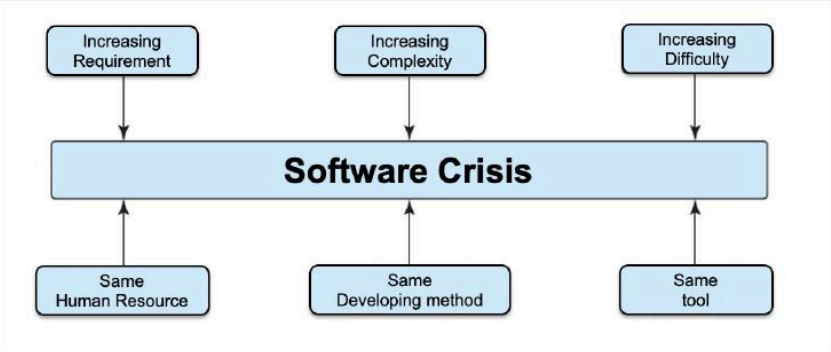
★ 시험 유지보수 비용 > 개발 비용

Etc of Software

Software crisis

Software crisis?

Software crisis is a term used in the early days of computing science for **the difficulty of writing useful and efficient computer programs** in the required time.



Source: <All about software engineering>. 생능출판사. 최은만, & wikipedia

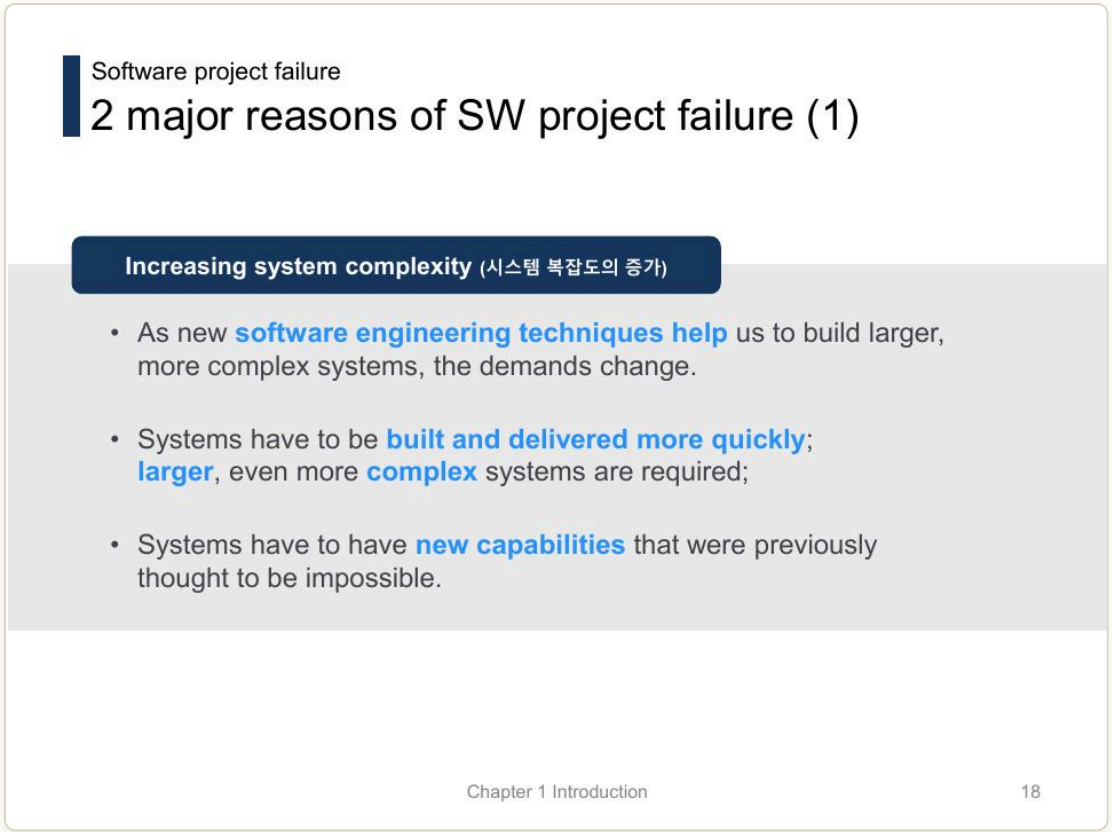
Chapter 1 Introduction 17

SW 위기 (Software crisis)

- 컴퓨터 과학 초창기에 쓰인 말
  - 쓸모 있고 효율적인 프로그램을 **정해진 시간 안에** 만들기 어려웠던 상황

✍ 메모

— 넘어가도 되는 쪽

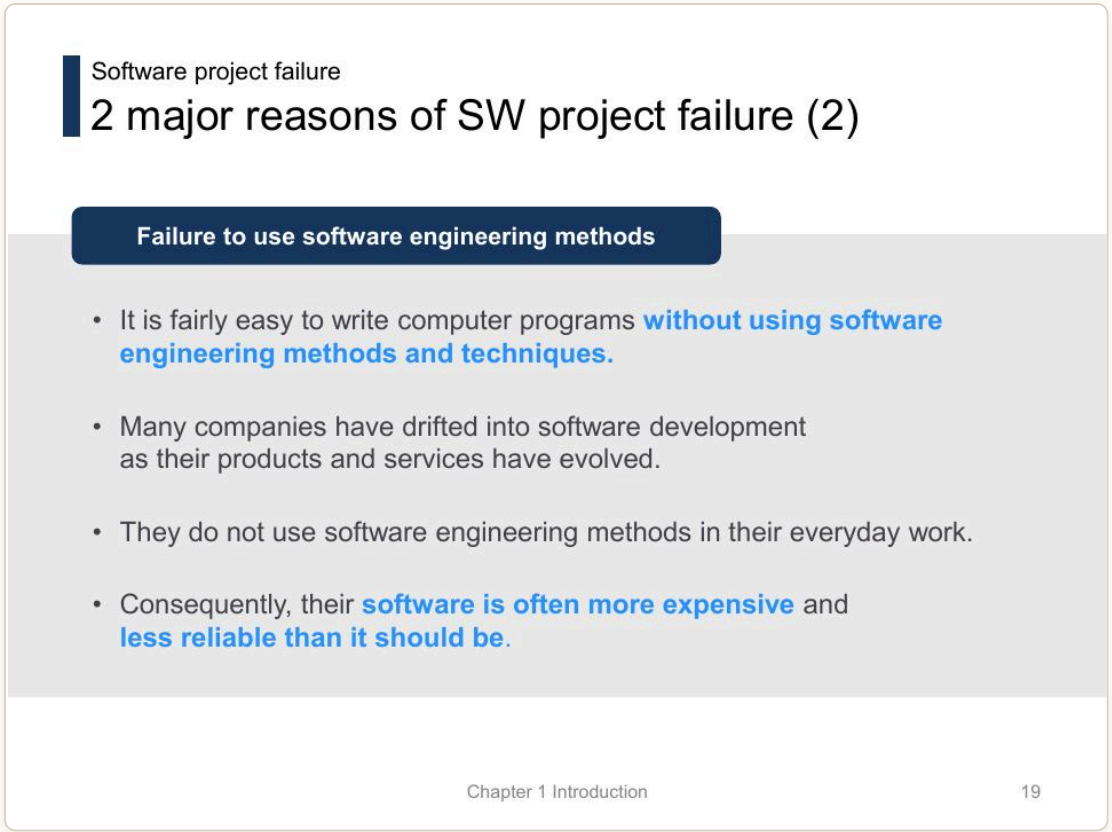


SW 프로젝트 실패의 두 원인 ①

- **시스템 복잡도의 증가 (Increasing system complexity)**
  - 새 SE 기법으로 더 크고 복잡한 시스템을 만들 수 있게 되면, 요구도 바뀐다
  - 더 빨리 만들어 내야 하고, 더 크고 복잡한 시스템이 요구됨
  - 예전엔 불가능하다던 기능까지 요구됨

✍️ 메모

💡 이해 잘 만들게 될수록 사람들은 더 어려운 걸 요구한다

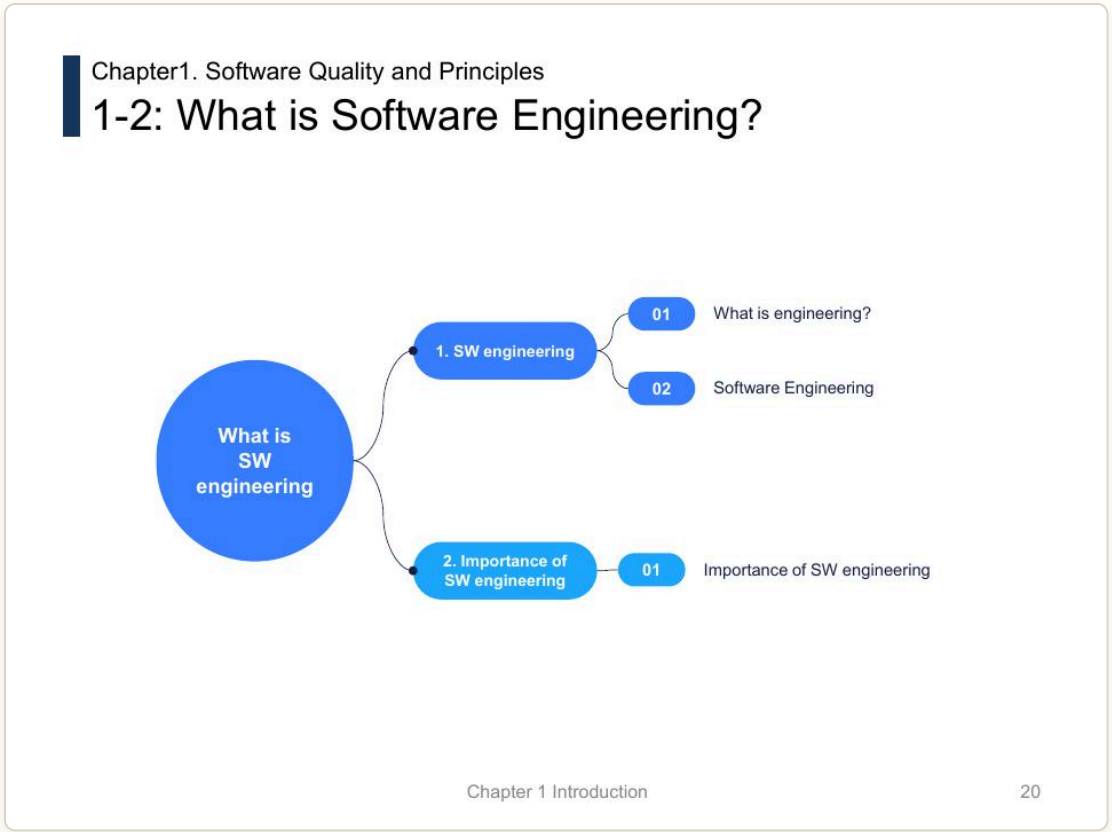


SW 프로젝트 실패의 두 원인 ②

- **SE 방법을 쓰지 않음 (Failure to use SE methods)**
  - SE 방법 없이도 프로그램을 쓰기는 꽤 쉽다
  - 많은 회사가 제품·서비스가 발전하며 어쩌다 SW 개발로 흘러들어옴
  - 일상 업무에서 SE 방법을 쓰지 않는다
  - → SW가 필요 이상으로 **비싸고 덜 신뢰할 만함**

✍ 메모

★ 시험 실패 원인 2개: 복잡도 증가 · SE 방법 미사용



1-2: 소프트웨어 공학이란?

- 공학이란? / 소프트웨어 공학
- SE의 중요성

✍️ 메모

💡 이해 절 표지 — 아래 쪽들의 지도

SW engineering

What is engineering?

- the study of using **scientific principles** to design and build machines, structures, and other things, including bridges, roads, vehicles, and buildings:
- Developing engineering accumulate skills to solve problems, and it is developed **engineering principles**
- Applying engineering principles make **procedures** and **standards** to solve specific problems

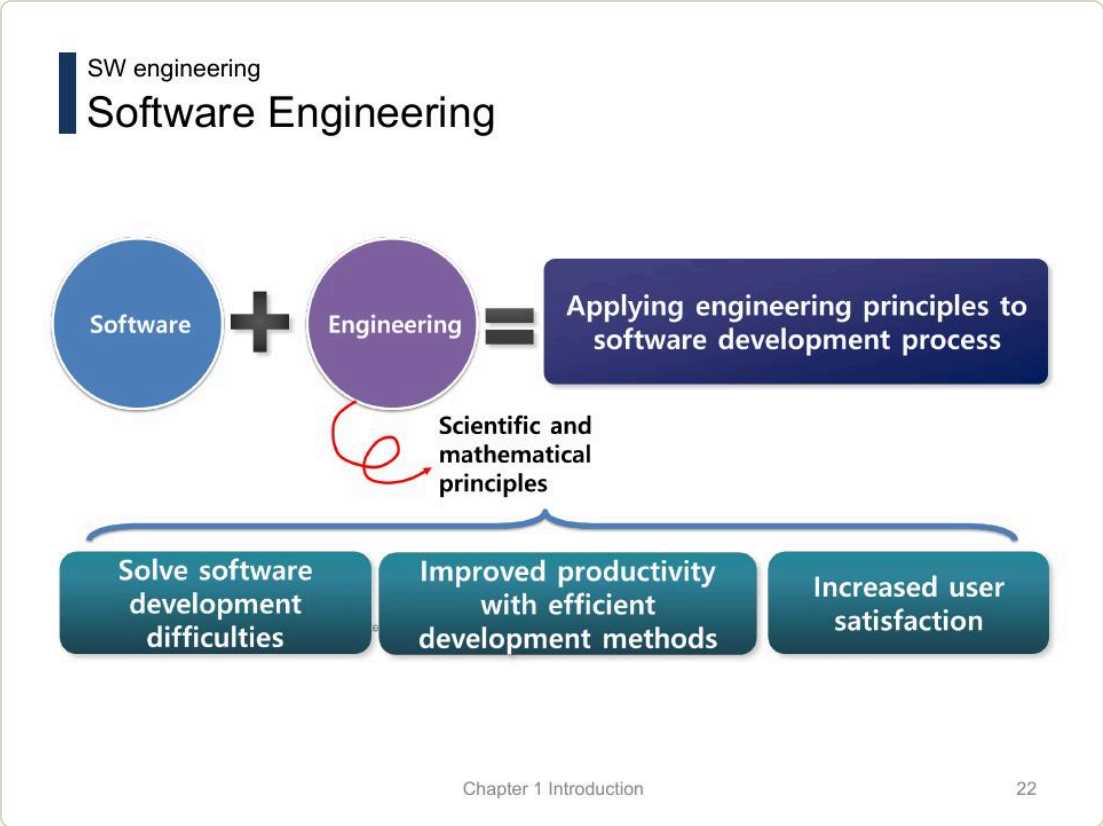
Chapter 1 Introduction 21

공학이란?

- 과학적 원리를 써서 기계·구조물(다리·도로·차량·건물 ...)을 설계하고 만드는 학문
- 공학이 발전하며 문제를 푸는 기술이 쌓이고 **공학 원리**가 생긴다
- 공학 원리를 적용해 특정 문제를 푸는 **절차와 표준**을 만든다

✍ 메모

— 넘어가도 되는 쪽

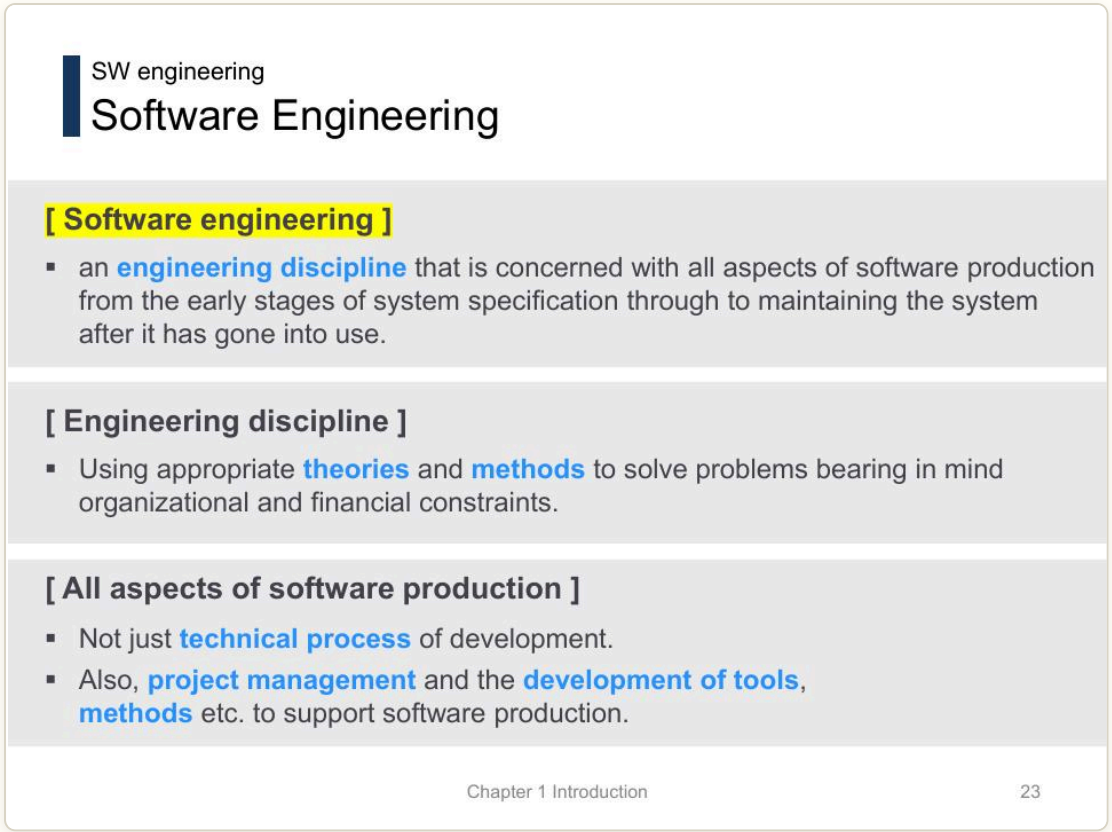


소프트웨어 공학 (그림)

- **Software + Engineering = SW 개발 과정에 공학 원리를 적용**
  - Engineering = 과학적·수학적 원리
- 효과: SW 개발의 어려움 해결 / 효율적 개발 방법으로 생산성 향상 / 사용자 만족도 향상

✍ 메모

★ 시험 효과 3개: 어려움 해결 · 생산성 · 사용자 만족



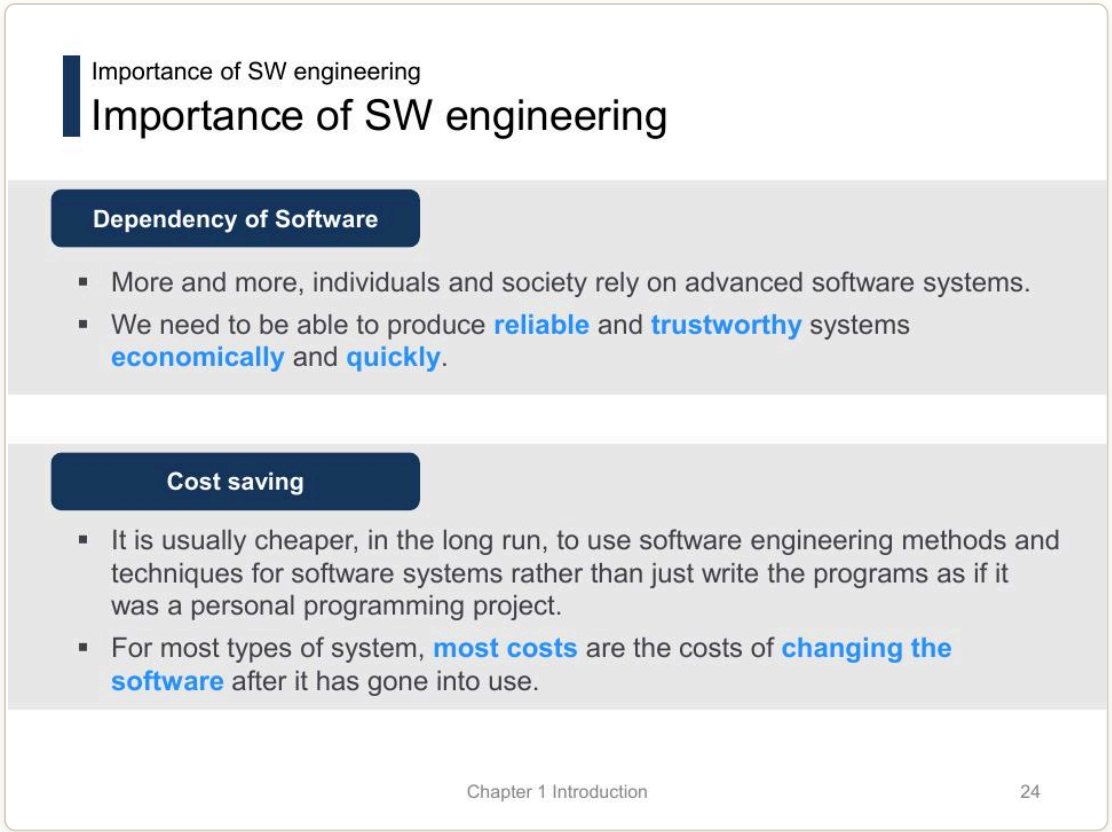
소프트웨어 공학의 정의

- **SE**: 시스템 명세 초기 단계부터 사용에 들어간 뒤 **유지보수까지**, SW 생산의 **모든 측면**을 다루는 공학 분야
- **공학 분야(engineering discipline)**: 조직적·재정적 **제약을 고려**하며 알맞은 이론과 방법으로 문제를 푼다
- **SW 생산의 모든 측면**: 기술적 개발 과정만이 아니라 **프로젝트 관리**, SW 생산을 돕는 도구·방법 개발까지

✍️ 메모

★ 시험 정의 영어: an engineering discipline concerned with all aspects of software production

⚠ 주의 '모든 측면' = 관리·도구까지 포함 (코딩만이 아님)

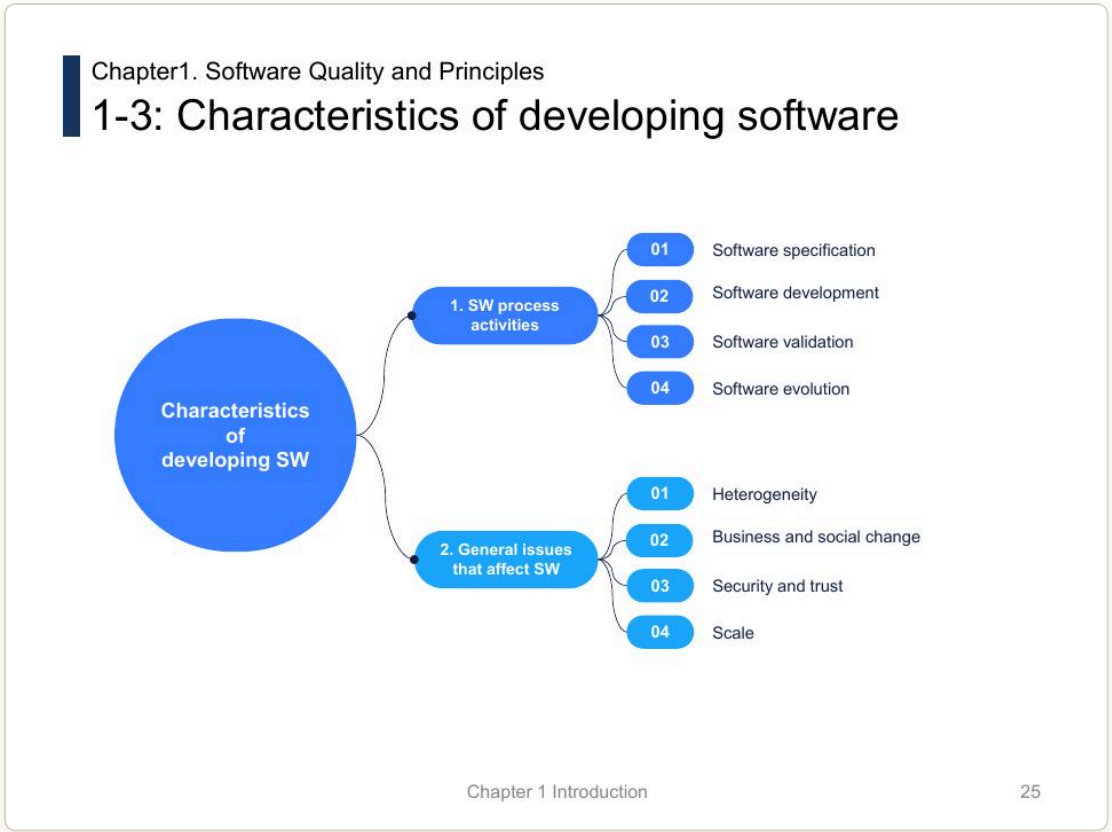


SE의 중요성

- **SW 의존도**: 개인과 사회가 점점 더 SW 시스템에 의존 → 믿을 수 있는 시스템을 경제적이고 빠르게 만들어야
- **비용 절감**: 개인 프로젝트처럼 그냥 짜는 것보다 SE 방법을 쓰는 게 **장기적으로 더 싸다**
  - 대부분 시스템에서 비용의 대부분은 **사용 후 변경** 비용

✍️ 메모

★ 시험 중요성 2개: 의존도(신뢰) · 장기 비용 절감

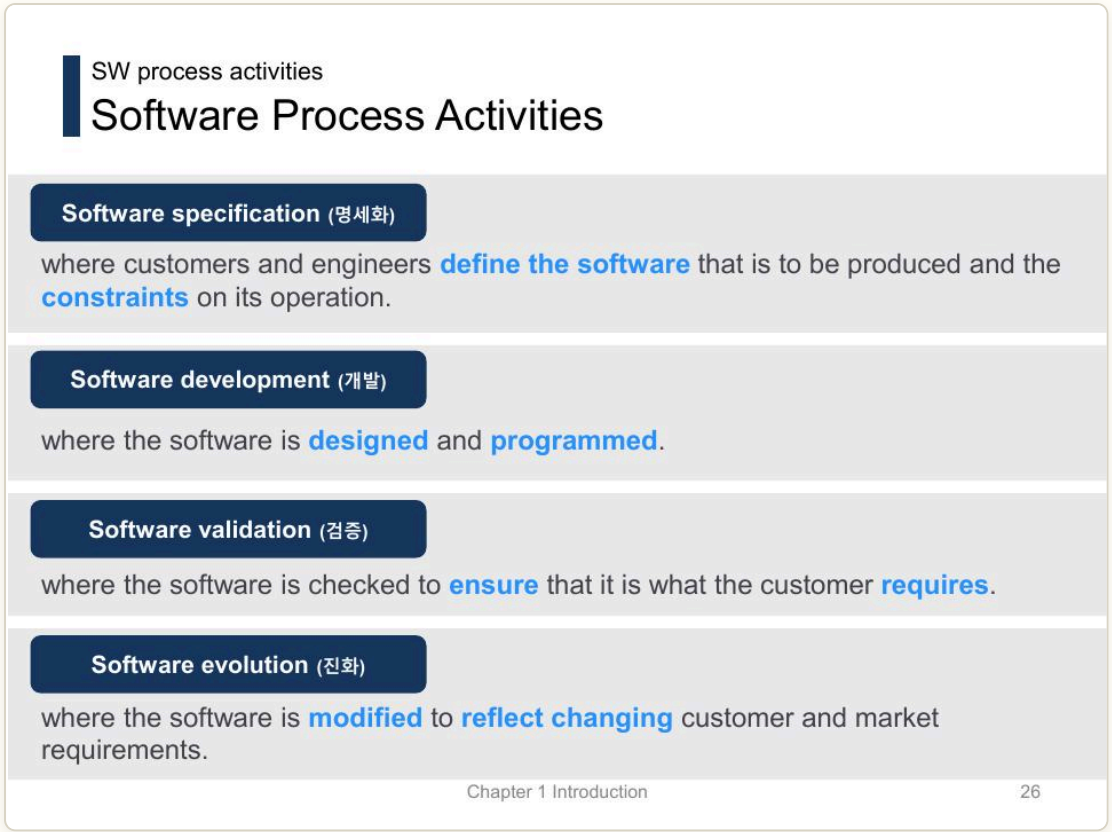


1-3: SW 개발의 특징

- SW 프로세스 활동 — 명세화, 개발, 검증, 진화
- SW에 영향을 주는 일반 이슈 — 이질성, 비즈니스·사회 변화, 보안과 신뢰, 규모

✍️ 메모

💡 이해 절 표지 — 아래 쪽들의 지도

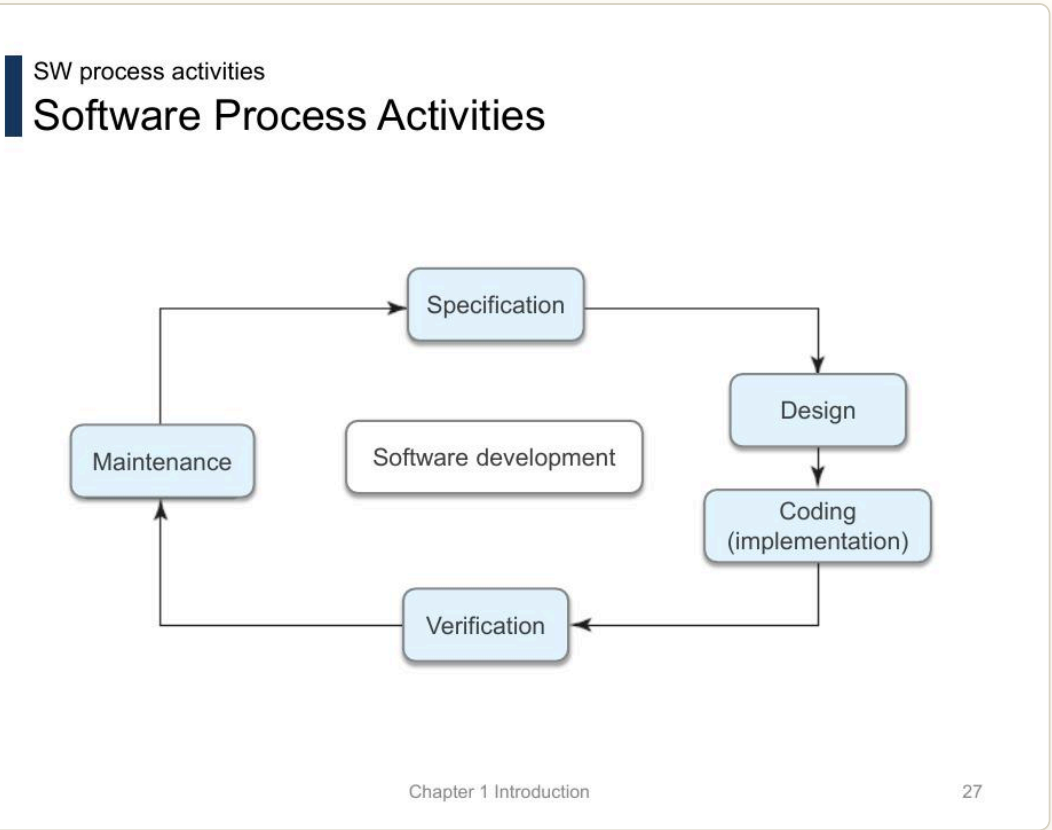


SW 프로세스 활동

- **Software specification (명세화)** — 고객과 엔지니어가 만들 SW와 운영 제약을 정의
- **Software development (개발)** — SW를 설계하고 프로그래밍
- **Software validation (검증)** — 고객이 원하는 것이 맞는지 확인
- **Software evolution (진화)** — 바뀌는 고객·시장 요구에 맞게 수정

✍️ 메모

★ 시험 4대 활동 영어로 순서대로: Specification · Development · Validation · Evolution

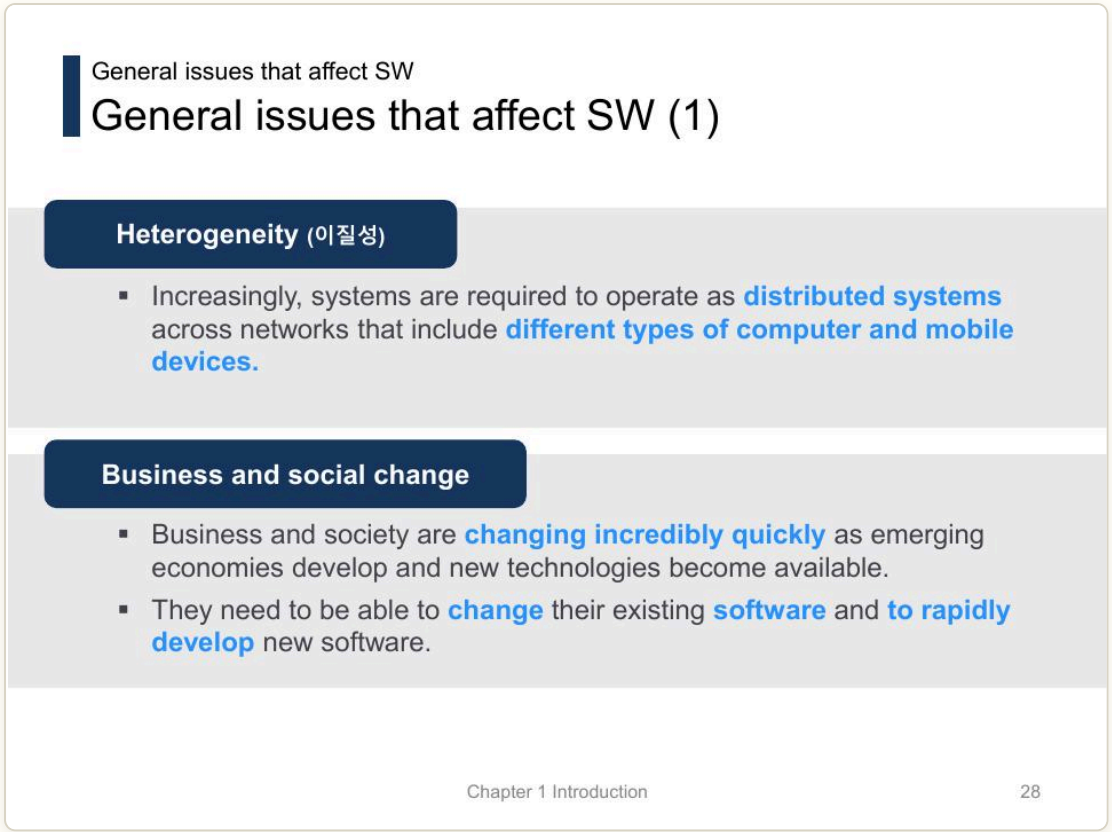


SW 프로세스 활동 (그림)

- 명세(Specification) → 설계(Design) → 구현(Coding) → 검증(Verification) → 유지보수(Maintenance)
  - 설계·구현이 'Software development'에 해당

✍ 메모

— 넘어가도 되는 쪽

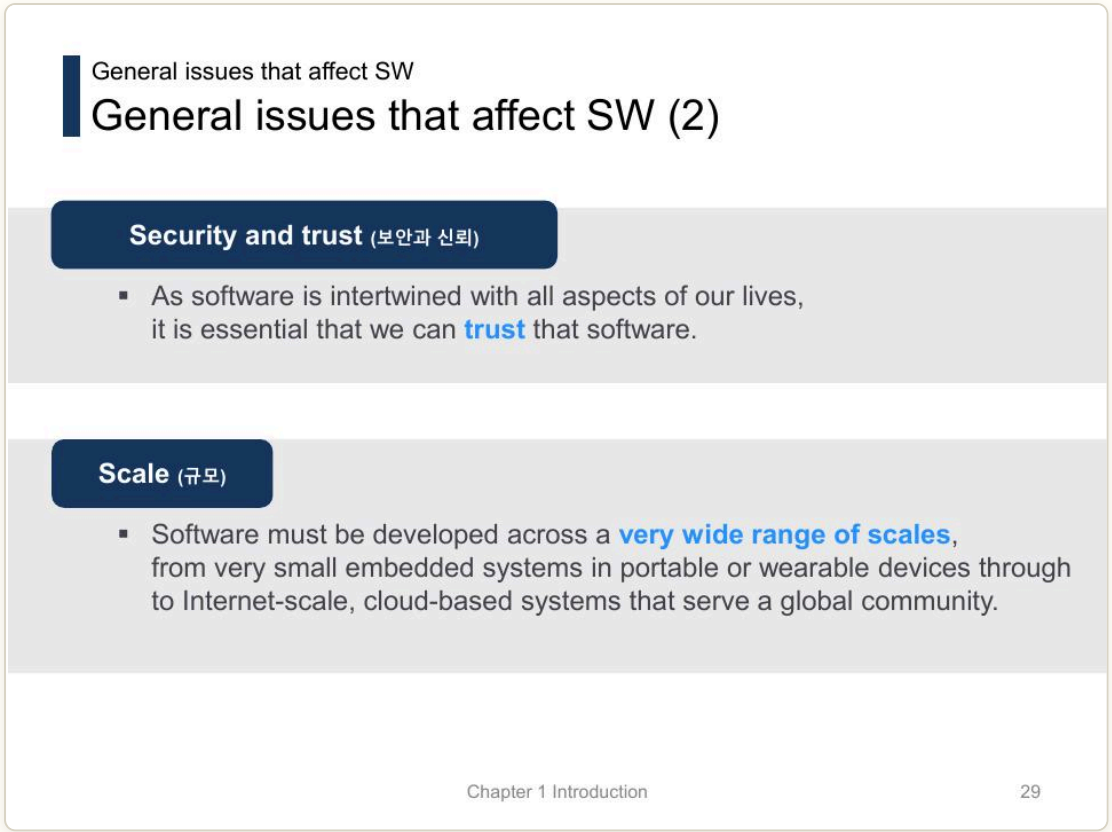


SW에 영향을 주는 일반 이슈 ①

- **Heterogeneity (이질성)**
  - 시스템이 다른 종류의 컴퓨터·모바일 기기를 포함한 네트워크에서 **분산 시스템**으로 동작해야
- **Business and social change (비즈니스·사회 변화)**
  - 신흥 경제와 새 기술로 비즈니스와 사회가 **아주 빠르게** 변화
  - 기존 SW를 바꾸고 새 SW를 빨리 개발할 수 있어야

✍ 메모

— 넘어가도 되는 쪽

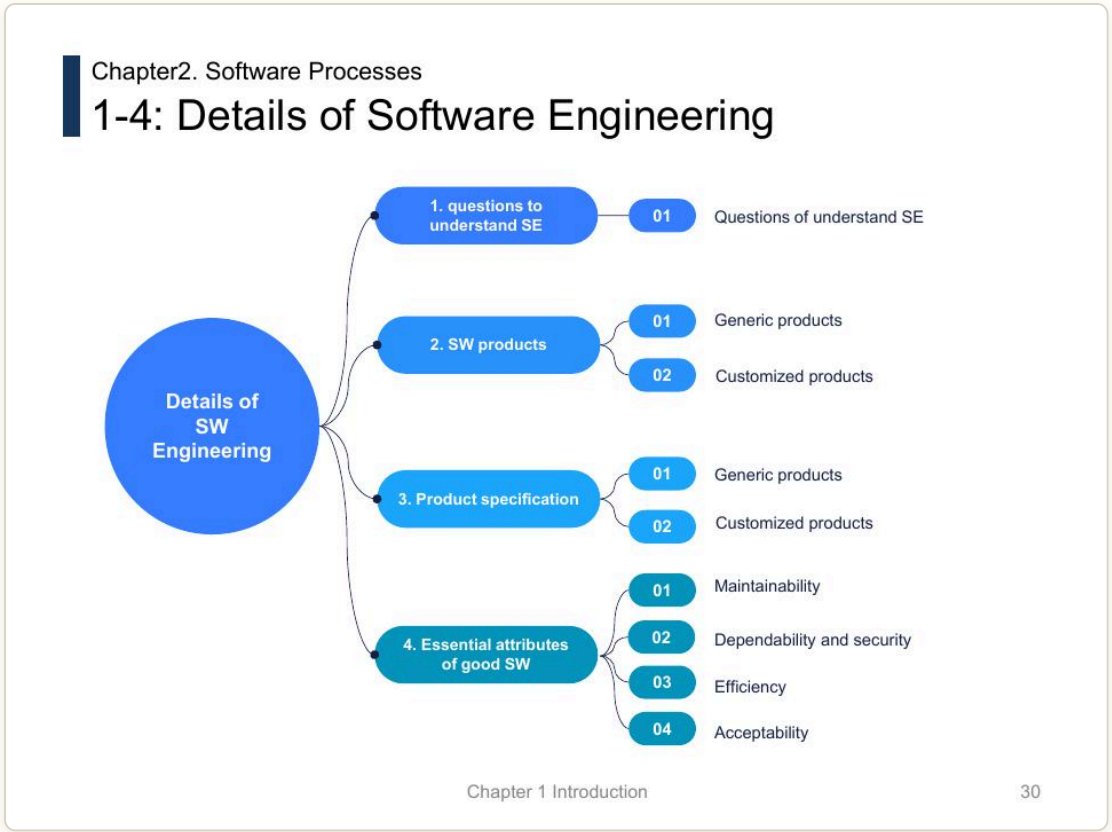


SW에 영향을 주는 일반 이슈 ②

- **Security and trust (보안과 신뢰)**
  - SW가 삶의 모든 면에 얽혀 있으니 **믿을 수 있어야** 한다
- **Scale (규모)**
  - 휴대·웨어러블의 아주 작은 임베디드부터 전 세계를 대상으로 하는 인터넷 규모 클라우드까지

✍ 메모

★ 시험 일반 이슈 4개: Heterogeneity · Business and social change · Security and trust · Scale

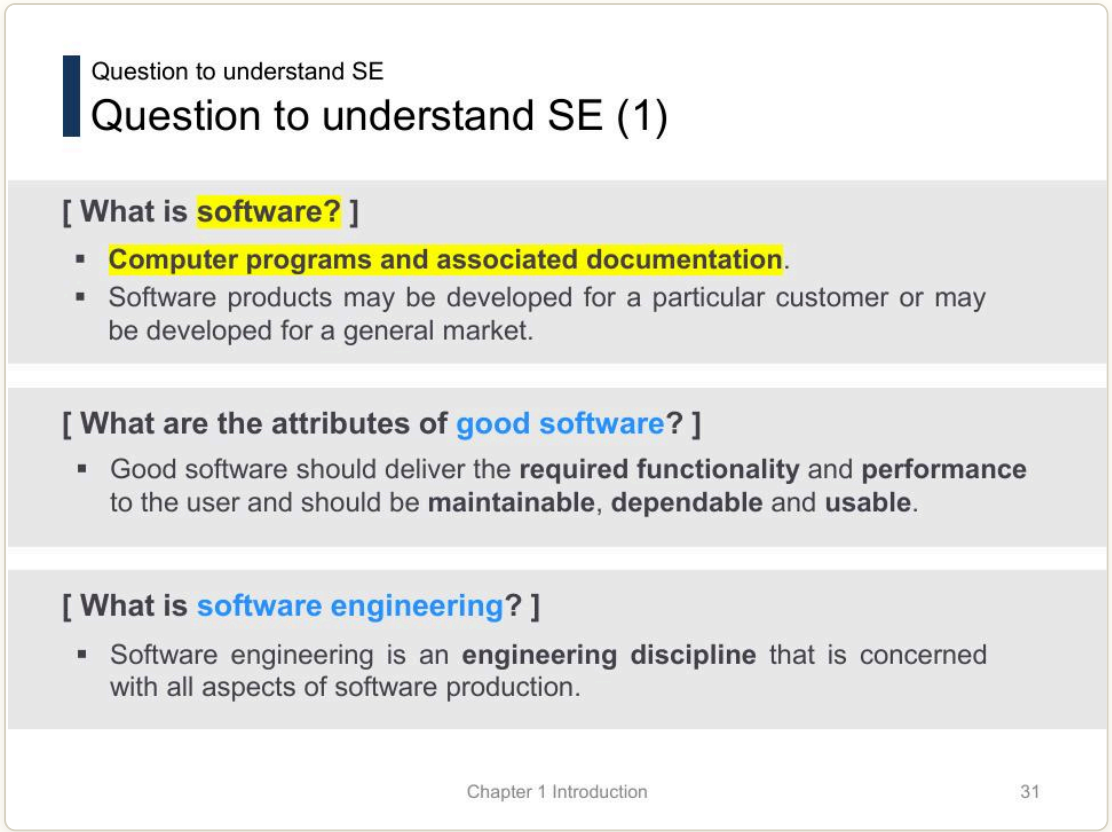


1-4: SE의 세부

- SE를 이해하는 질문들
- SW 제품 — 일반 제품 / 맞춤형 제품
- 제품 명세
- 좋은 SW의 필수 속성 — 유지보수성, 확실성과 보안성, 효율성, 수용성

✍️ 메모

💡 이해 절 표지 — 아래 쪽들의 지도



SE를 이해하는 질문 ①

- **SW란?** — 컴퓨터 프로그램과 관련 문서
  - 특정 고객용으로, 또는 일반 시장용으로 개발
- **좋은 SW의 속성은?** — 필요한 기능과 성능을 주고, **유지보수 가능·믿을 수 있고·쓸 만해야**
- **SE란?** — SW 생산의 모든 측면을 다루는 공학 분야

✍️ 메모

— 넘어가도 되는 쪽

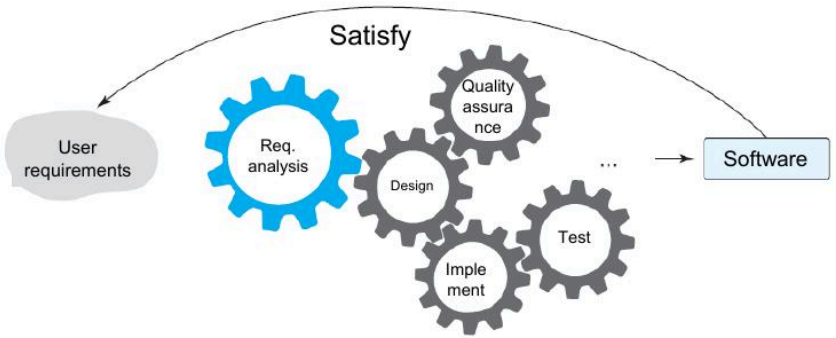
Question to understand SE

Question to understand SE (2)

[What are the fundamental software engineering activities?]

- Software specification, software development, software validation and software evolution.

Source: <All about software engineering>. 생능출판사. 최은만



Chapter 1 Introduction 32

SE를 이해하는 질문 ②

- **SE의 기본 활동은?** — SW **명세화, 개발, 검증, 진화**
- 그림: 사용자 요구 → 요구 분석 → 설계 → 구현 → 테스트, 품질 보증으로 만족

✍ 메모

— 넘어가도 되는 쪽

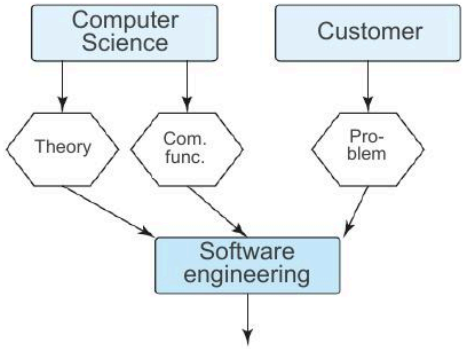
Question to understand SE

Question to understand SE (3)

[What is the difference between software engineering and computer science?]

- Computer science focuses on **theory** and **fundamentals**;
- Software engineering is concerned with the **practicalities** of developing and delivering useful software.

Source: <All about software engineering>. 생능출판사. 최은만



Chapter 1 Introduction 33

SE를 이해하는 질문 ③ — SE vs CS

- **SE와 컴퓨터 과학(CS)의 차이는?**
  - CS: **이론과 기초**에 집중
  - SE: 쓸모 있는 SW를 개발·전달하는 **실제(practicalities)**에 집중

✍️ 메모

★ 시험 **CS = theory and fundamentals / SE = practicalities**

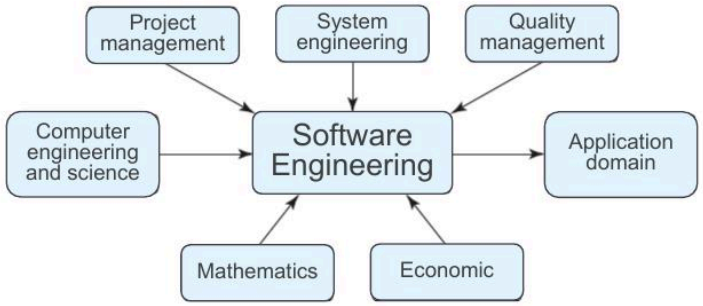
Question to understand SE

Question to understand SE (4)

[What is the **difference** between **software engineering** and **system engineering**?]

- **System engineering** is concerned with all aspects of computer-based systems development including **hardware, software** and **process engineering**.
- **Software engineering** is part of this **more general process**.

Source: <All about software engineering>. 생능출판사. 최은만



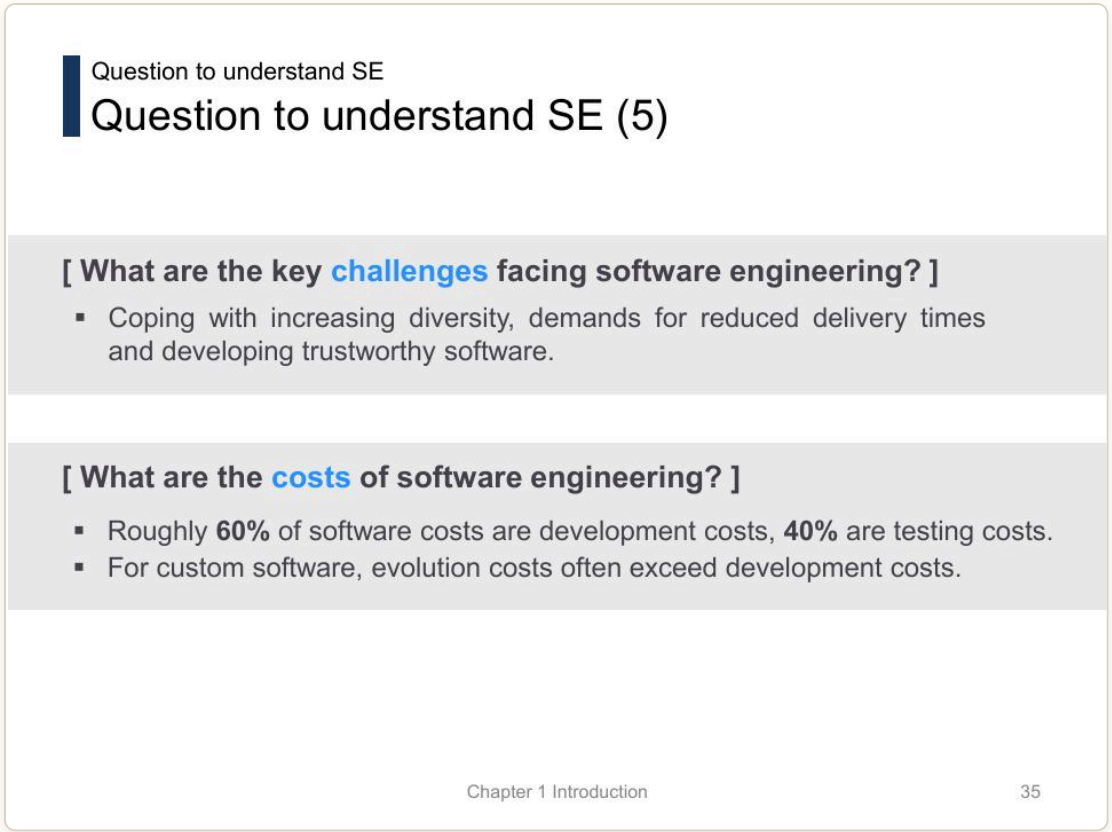
Chapter 1 Introduction 34

SE를 이해하는 질문 ④ — SE vs 시스템 공학

- **SE와 시스템 공학의 차이는?**
  - 시스템 공학: HW·SW·프로세스 공학을 포함한 컴퓨터 기반 시스템 개발의 **모든 측면**
  - SE는 이 더 일반적인 과정의 **일부**

✍ 메모

★ 시험 System engineering ⊃ Software engineering

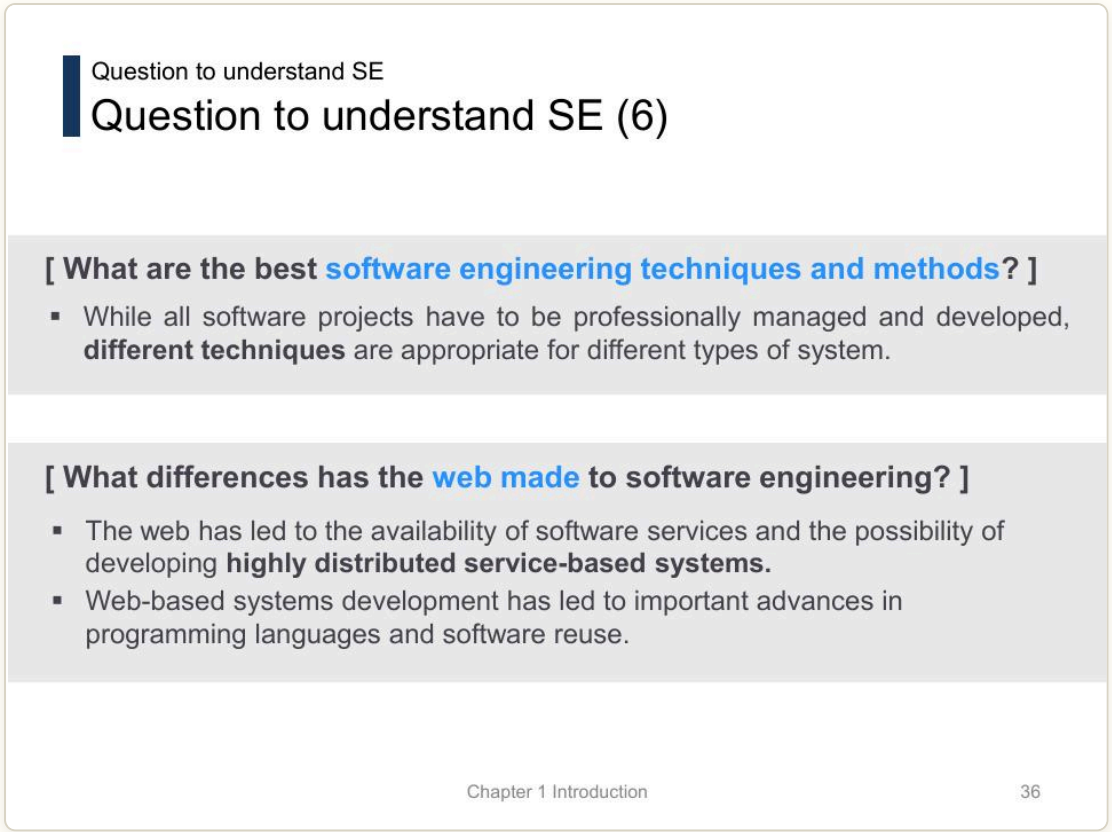


SE를 이해하는 질문 ⑤

- **SE의 핵심 과제는?** — 늘어나는 다양성, 더 짧은 납기 요구, 믿을 수 있는 SW 개발
- **SE 비용은?**
  - 대략 **60%는 개발**, **40%는 테스트** 비용
  - 맞춤형 SW는 **진화(유지보수) 비용이 개발 비용을 넘는** 경우가 많다

✍️ 메모

★ 시험 숫자: 개발 60% / 테스트 40%

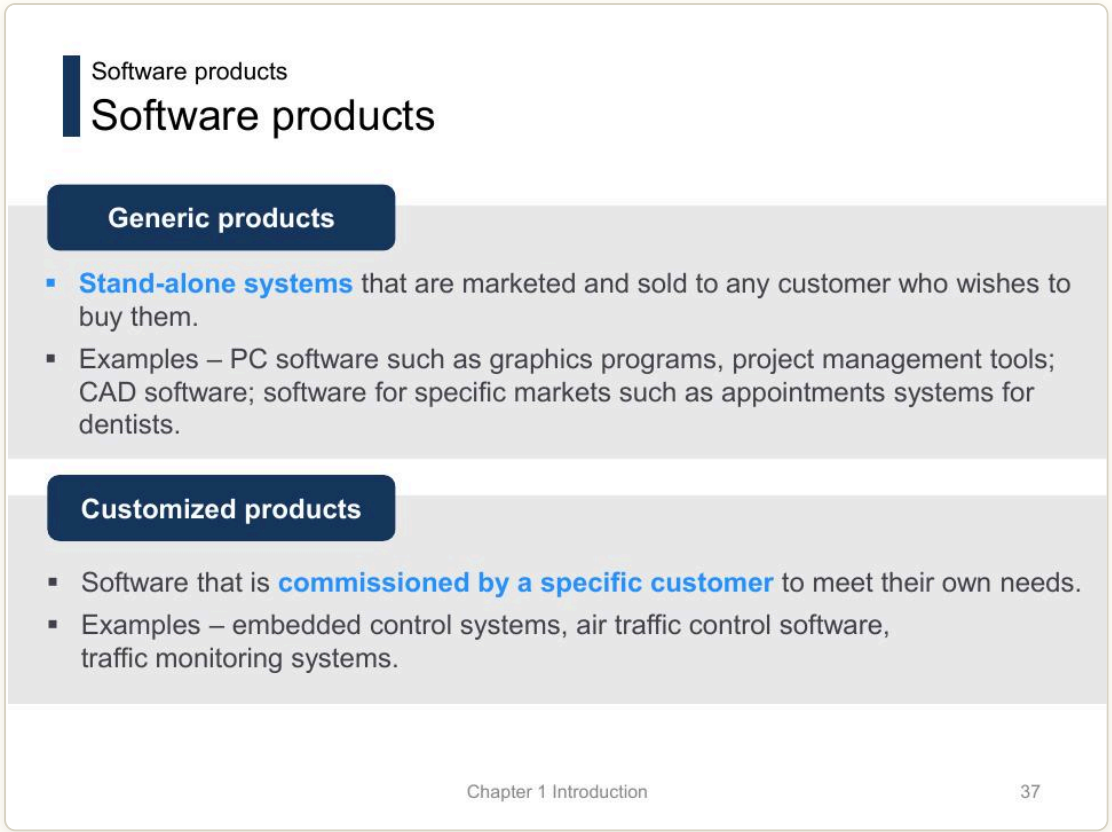


SE를 이해하는 질문 ⑥

- **가장 좋은 SE 기법과 방법은?**
  - 모든 프로젝트는 전문적으로 관리·개발돼야 하지만, 시스템 종류마다 **알맞은 기법이 다르다**
- **웹은 SE에 어떤 차이를 만들었나?**
  - SW 서비스와 고도로 분산된 서비스 기반 시스템이 가능해짐
  - 웹 기반 개발이 프로그래밍 언어와 **SW 재사용**에 큰 발전을 가져옴

✍️ 메모

⚠ 주의 '만능 기법은 없다' — 시스템마다 다르다

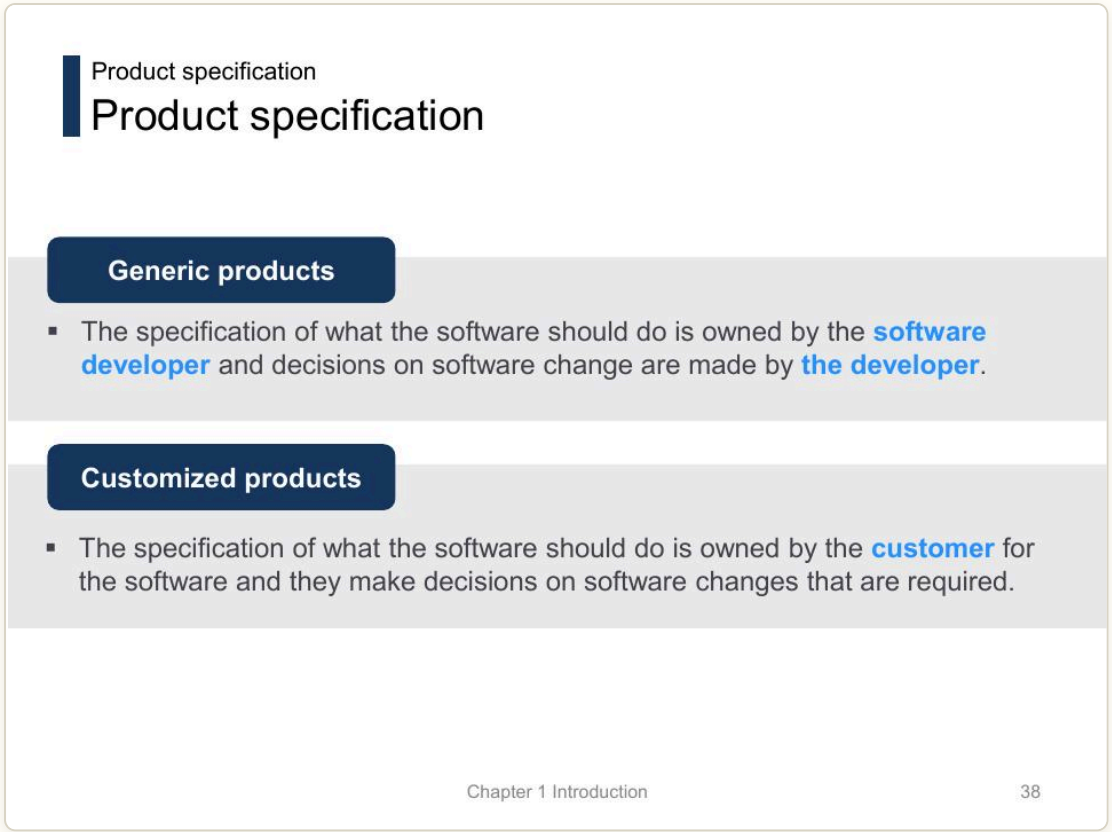


SW 제품

- **Generic products (일반 제품)**
  - 원하는 누구에게나 판매하는 독립 시스템
  - 예: 그래픽 프로그램, 프로젝트 관리 도구, CAD, 치과 예약 시스템처럼 특정 시장용 SW
- **Customized products (맞춤형 제품)**
  - 특정 고객이 자기 필요에 맞게 의뢰한 SW
  - 예: 임베디드 제어 시스템, 항공 교통 관제, 교통 감시 시스템

✍️ 메모

— 넘어가도 되는 쪽

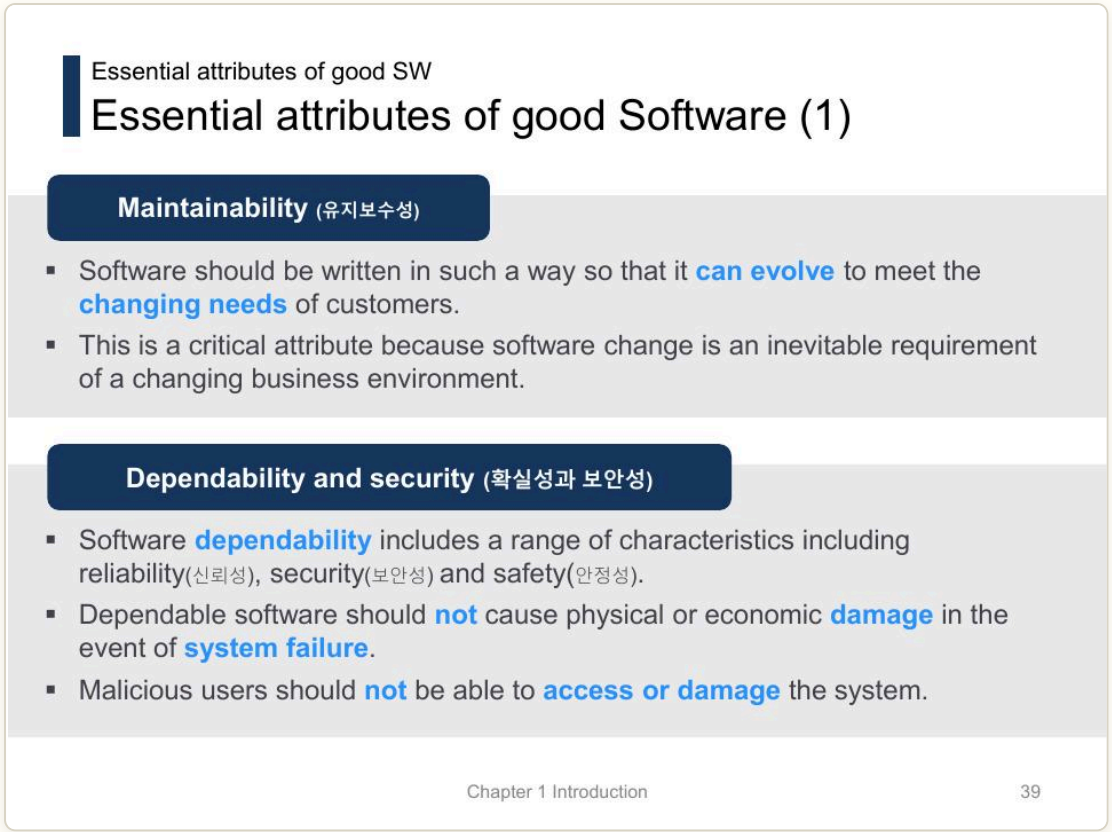


제품 명세

- **Generic**: 무엇을 할지에 대한 명세는 **개발자** 소유, 변경 결정도 개발자가
- **Customized**: 명세는 **고객** 소유, 필요한 변경도 고객이 결정

✍ 메모

★ 시험 명세 소유자: 일반 = 개발자 / 맞춤 = 고객

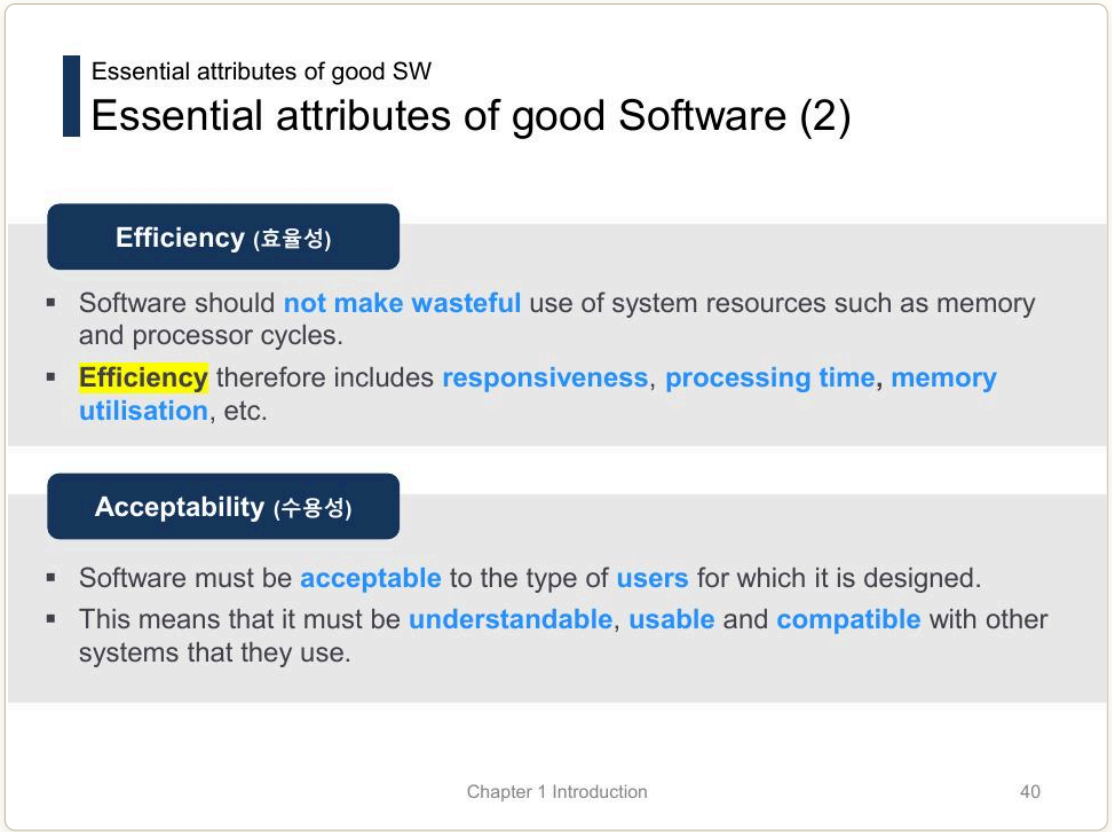


좋은 SW의 필수 속성 ①

- **Maintainability (유지보수성)**
  - 고객의 바뀌는 요구에 맞춰 **진화**할 수 있게 작성
  - 비즈니스 환경이 바뀌면 SW 변경은 피할 수 없으니 **매우 중요**
- **Dependability and security (확실성과 보안성)**
  - 신뢰성(reliability), 보안성(security), 안전성(safety) 포함
  - 시스템 고장 시 물리적·경제적 피해를 주지 않아야
  - 악의적 사용자가 접근하거나 손상시킬 수 없어야

✍️ 메모

💡 이해 슬라이드 '안정성'은 safety(안전성)의 번역

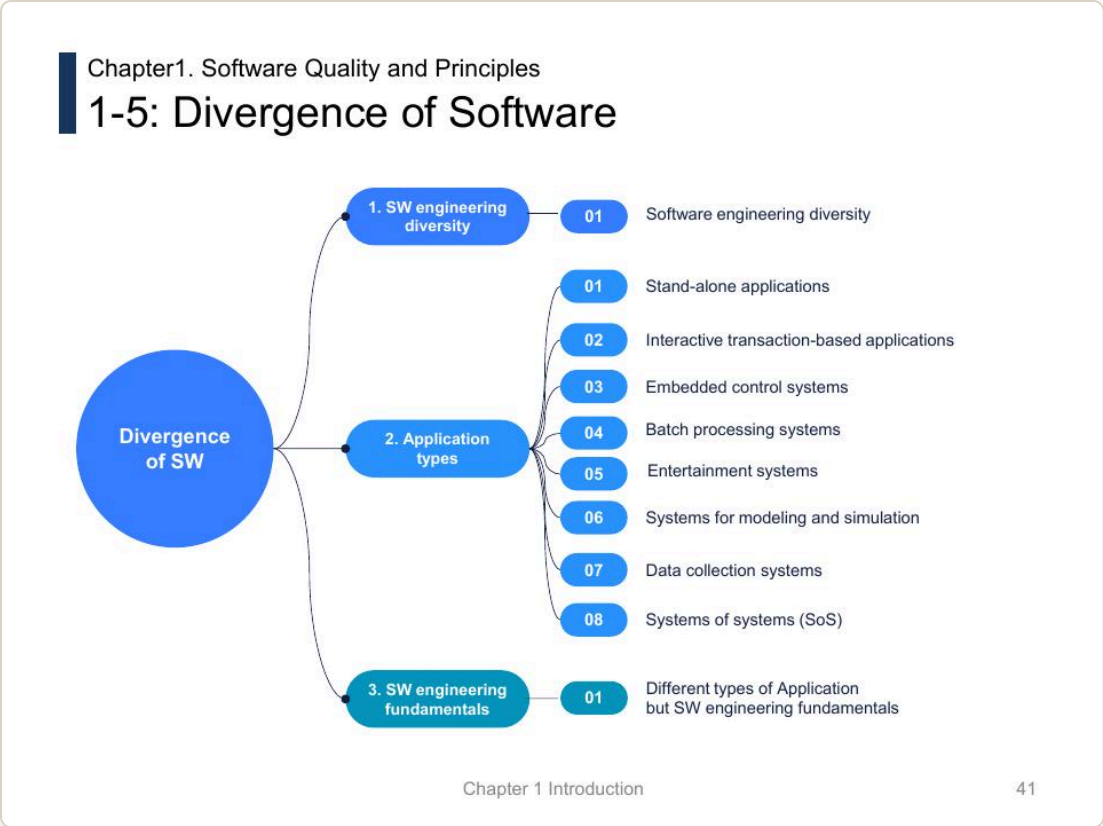


좋은 SW의 필수 속성 ②

- **Efficiency (효율성)**
  - 메모리·프로세서 같은 자원을 낭비하지 않아야
  - 응답성, 처리 시간, 메모리 사용 등
- **Acceptability (수용성)**
  - 대상 사용자가 받아들일 수 있어야
  - 이해하기 쉽고, 쓰기 쉽고, 쓰는 다른 시스템과 호환

✍️ 메모

★ 시험 필수 속성 4개: Maintainability · Dependability & security · Efficiency · Acceptability (시험 1순위)

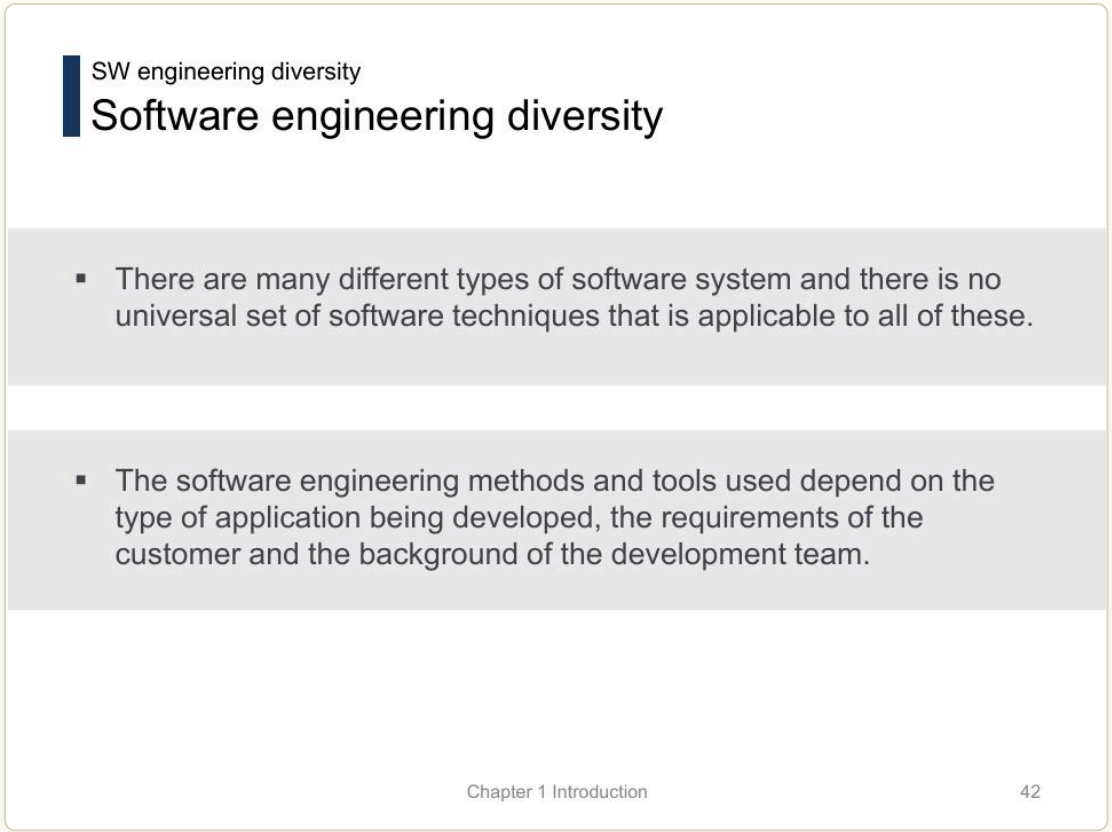


1-5: SW의 다양성

- SE 다양성
- 응용 유형 8가지
- 응용은 달라도 SE 기본은 같다

✍️ 메모

💡 이해 절 표지 — 아래 쪽들의 지도

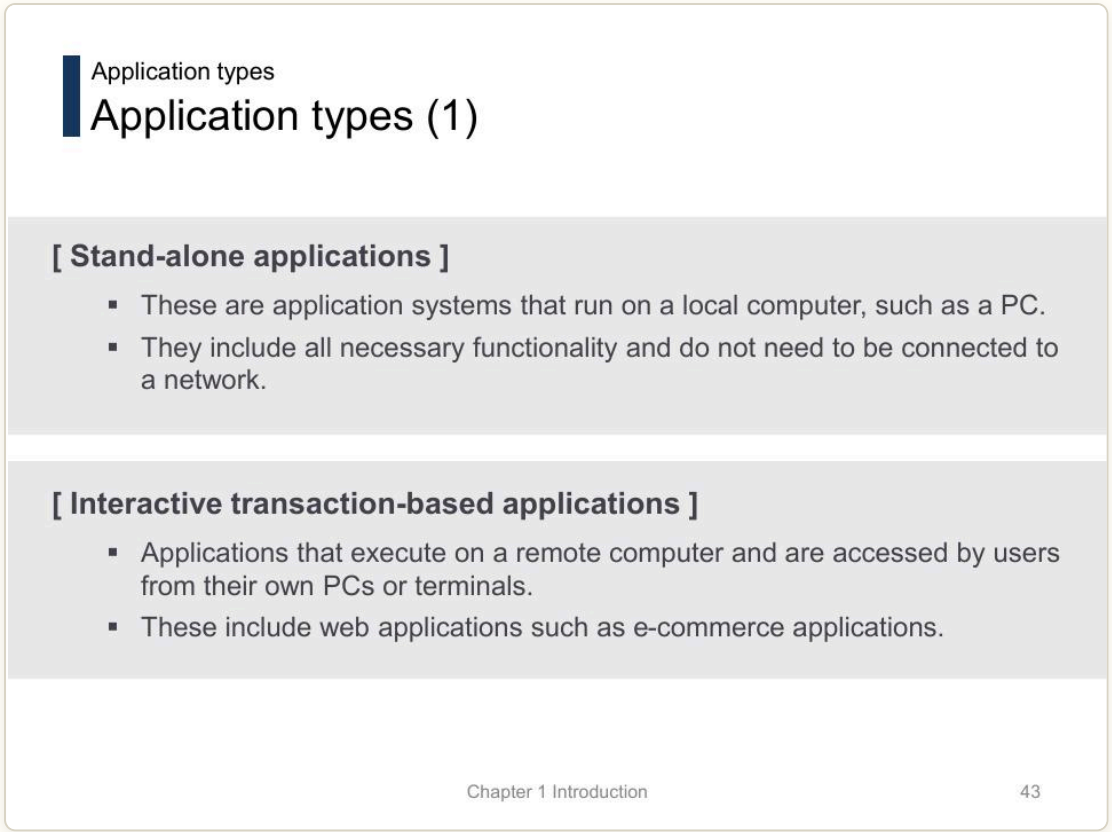


SE의 다양성

- SW 시스템 종류가 매우 많고, **모두에 통하는 보편적 기법은 없다**
- 쓰는 SE 방법과 도구는 응용 종류, 고객 요구, 개발 팀 배경에 따라 달라진다

✍️ 메모

— 넘어가도 되는 쪽

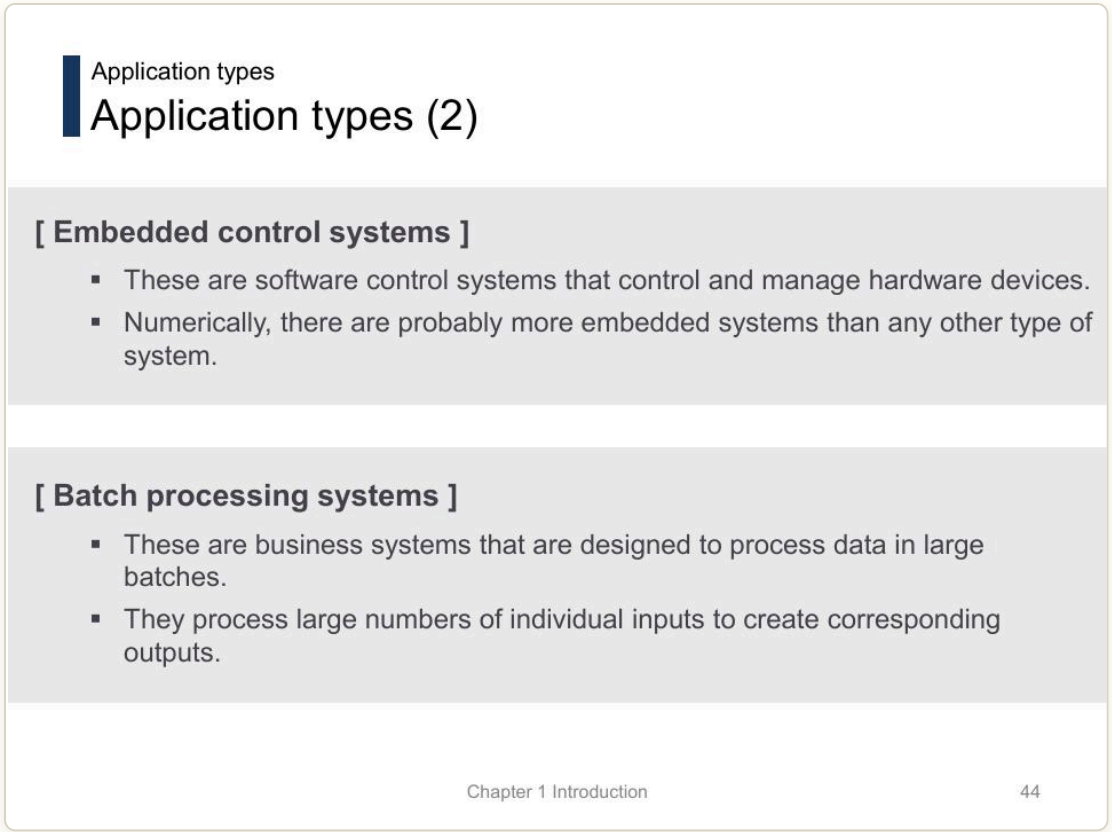


응용 유형 ①

- **Stand-alone applications** — PC 같은 로컬 컴퓨터에서 실행, 필요한 기능을 다 갖고 네트워크 연결 불필요
- **Interactive transaction-based applications** — 원격 컴퓨터에서 실행, 사용자는 자기 PC·단말로 접근
  - 예: 전자상거래 같은 웹 응용

✍️ 메모

— 넘어가도 되는 쪽

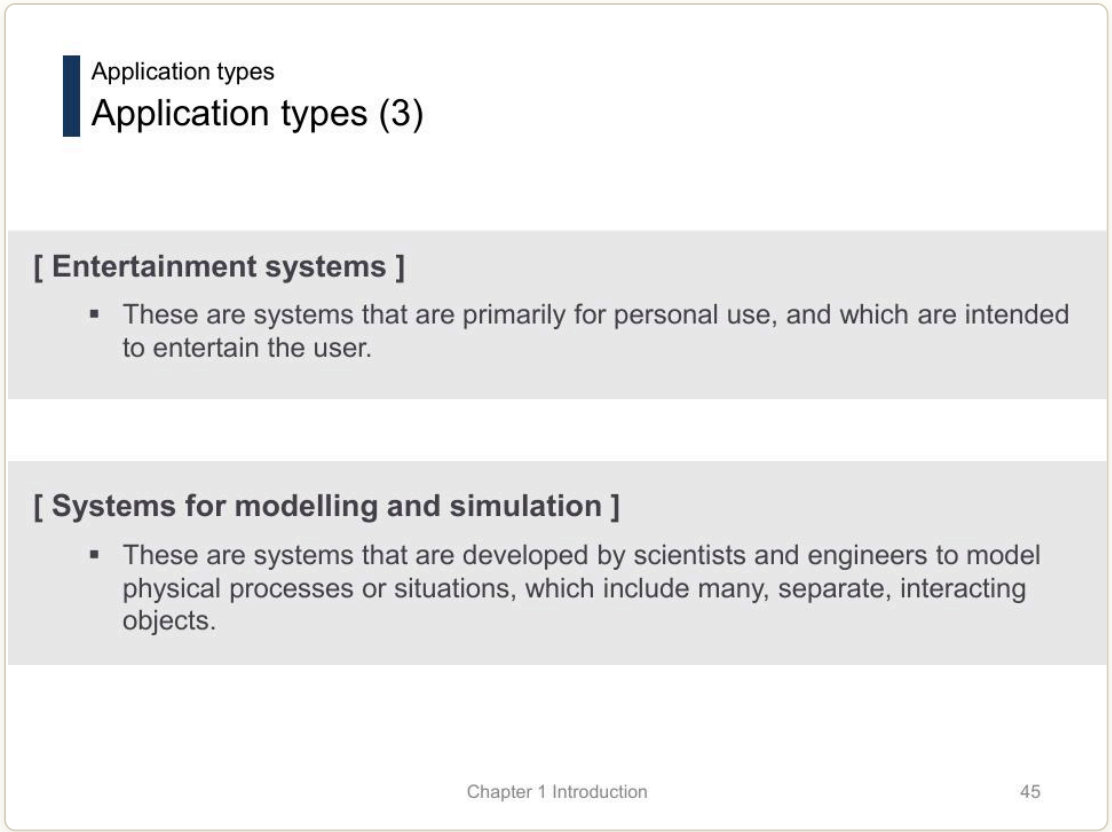


응용 유형 ②

- **Embedded control systems** — HW 장치를 제어·관리하는 SW
  - **숫자로는 아마 가장 많은** 시스템 유형
- **Batch processing systems** — 데이터를 대량 묶음(batch)으로 처리하는 업무 시스템
  - 많은 개별 입력을 처리해 대응하는 출력 생성

✍️ 메모

★ 시험 가장 많은 유형 = Embedded control systems

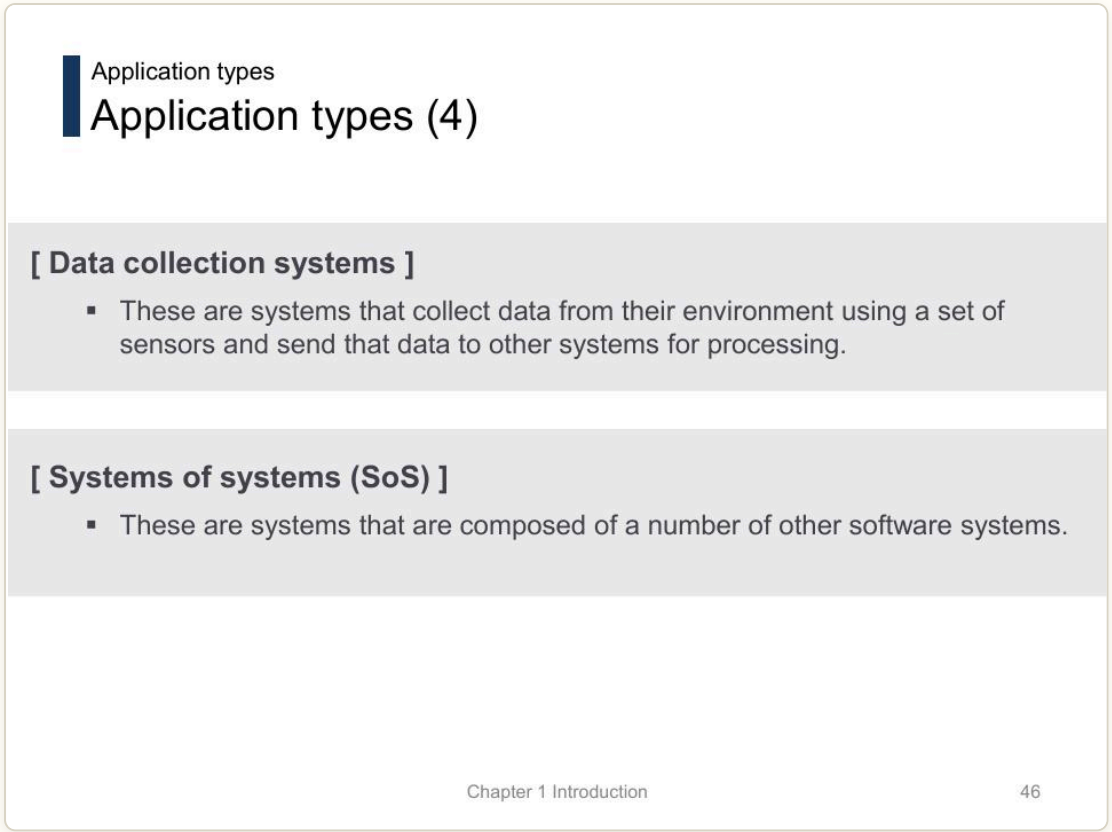


응용 유형 ③

- **Entertainment systems** — 주로 개인용, 사용자를 즐겁게 하려는 시스템
- **Systems for modelling and simulation** — 과학자·엔지니어가 물리 과정이나 상황을 모델링, 서로 작용하는 많은 객체를 포함

✍ 메모

— 넘어가도 되는 쪽

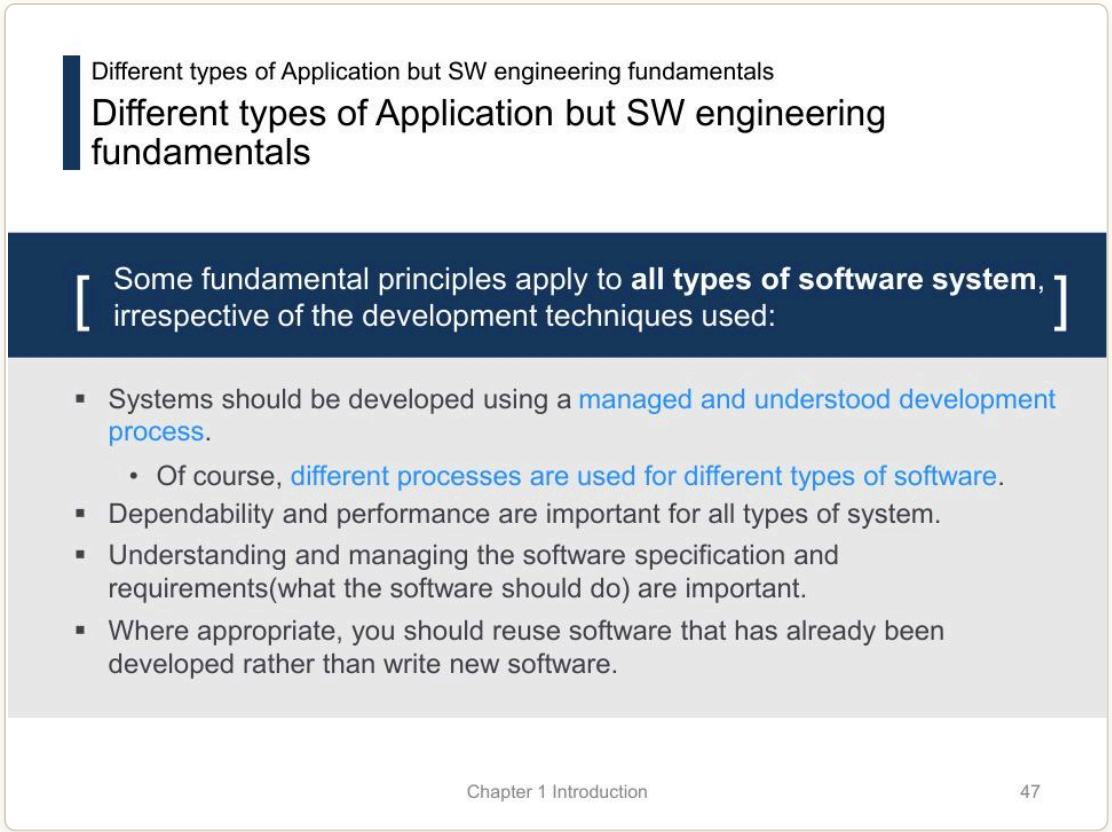


응용 유형 ④

- **Data collection systems** — 센서로 환경 데이터를 모아 다른 시스템에 보냄
- **Systems of systems (SoS)** — 여러 다른 SW 시스템으로 이루어진 시스템

✍️ 메모

★ 시험 응용 유형 8개 이름 외우기 (Stand-alone, Interactive transaction-based, Embedded control, Batch, Entertainment, Modeling & simulation, Data collection, SoS)

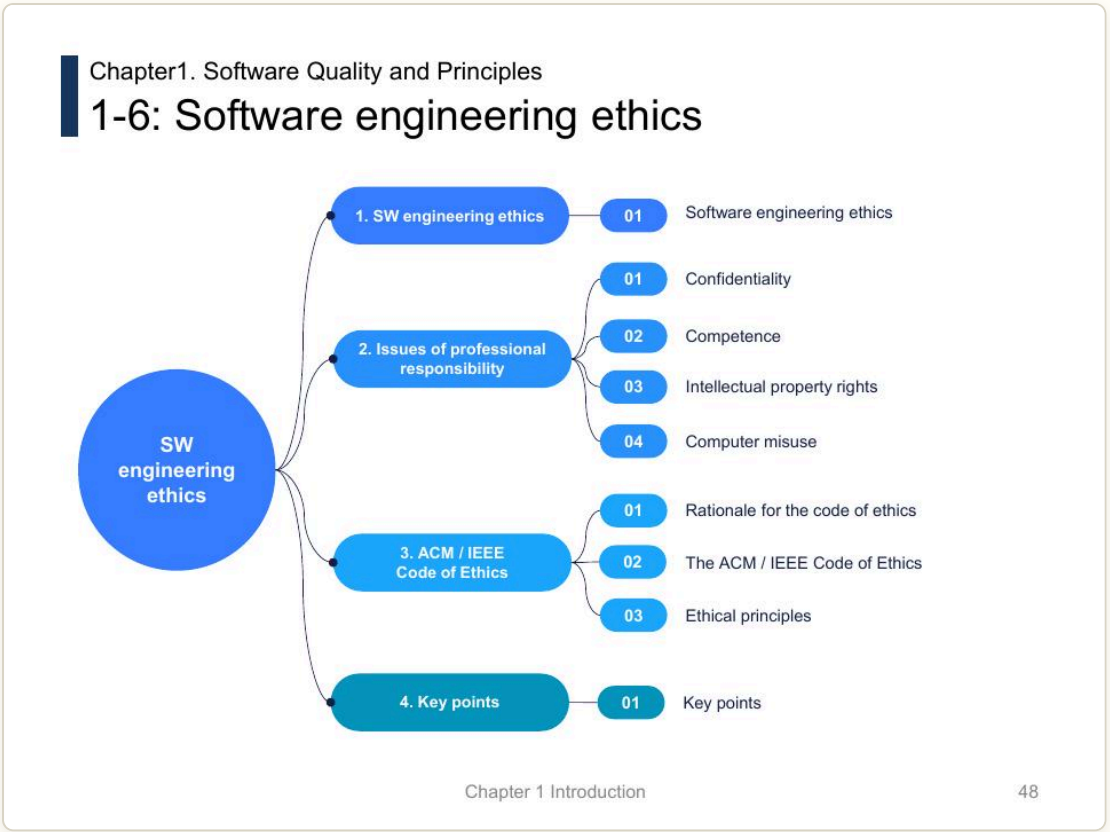


응용은 달라도 SE 기본은 같다

- 개발 기법과 상관없이 모든 SW 시스템에 통하는 기본 원리:
  - **관리되고 이해된 개발 프로세스**로 개발 (물론 종류마다 프로세스는 다름)
  - **확실성(dependability)과 성능**은 모든 시스템에 중요
  - SW **명세와 요구사항**(무엇을 할지)을 이해하고 관리하는 것이 중요
  - 알맞다면 새로 쓰기보다 이미 개발된 SW를 **재사용**

✍ 메모

★ 시험 기본 원리 4개: 관리된 프로세스 · 확실성과 성능 · 요구 관리 · 재사용

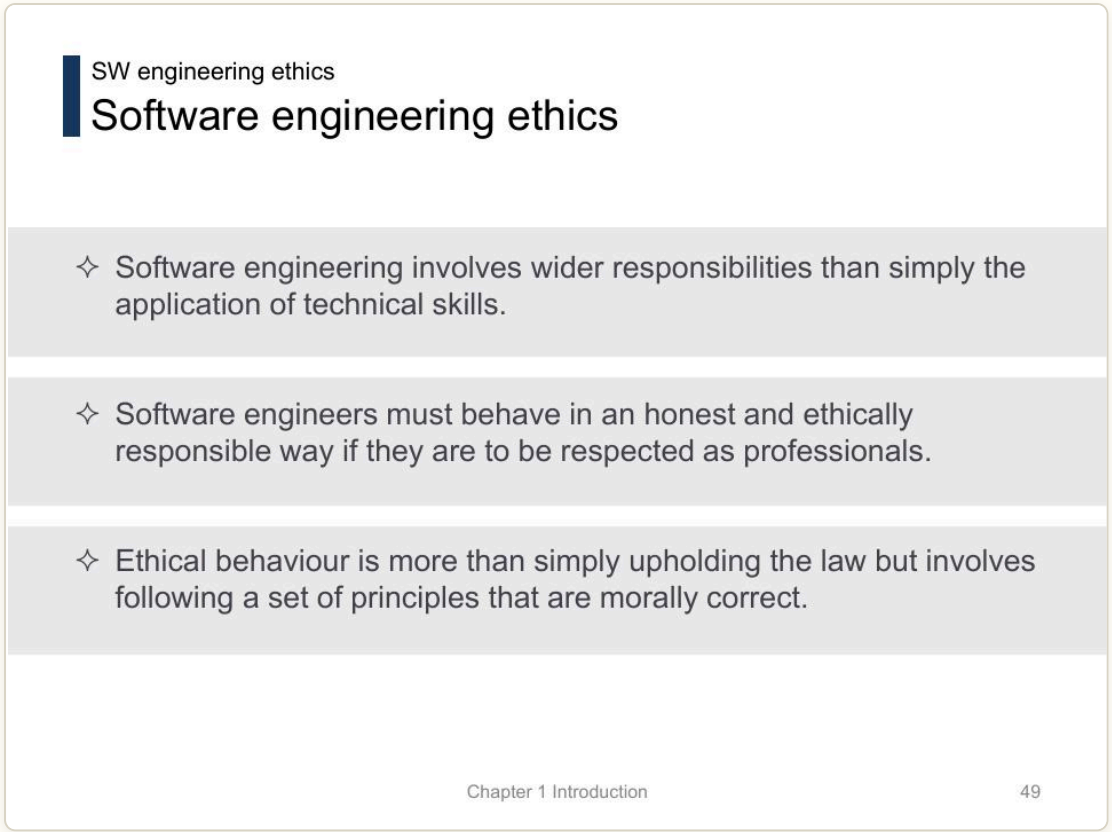


1-6: SE 윤리

- SE 윤리
- 전문가 책임 이슈 — 비밀유지, 능력, 지적재산권, 컴퓨터 남용
- ACM/IEEE 윤리 강령 — 근거, 강령, 원칙
- 핵심 정리

✍️ 메모

💡 이해 절 표지 — 아래 쪽들의 지도

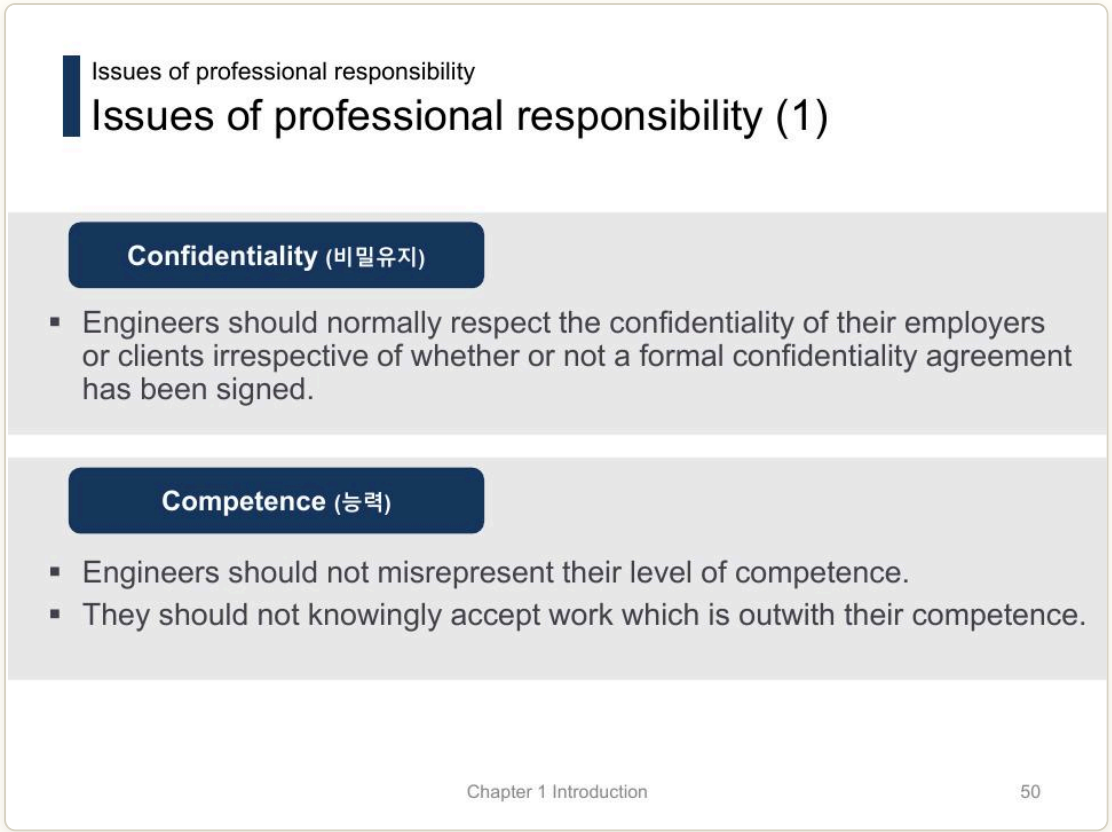


SE 윤리

- SE는 기술 적용보다 **더 넓은 책임**을 진다
- 전문가로 존중받으려면 **정직하고 윤리적으로** 행동해야
- 윤리적 행동은 **법을 지키는 것 이상** — 도덕적으로 옳은 원칙을 따르는 것

✍️ 메모

⚠ 주의 윤리 ≠ 단순 준법 (more than upholding the law)

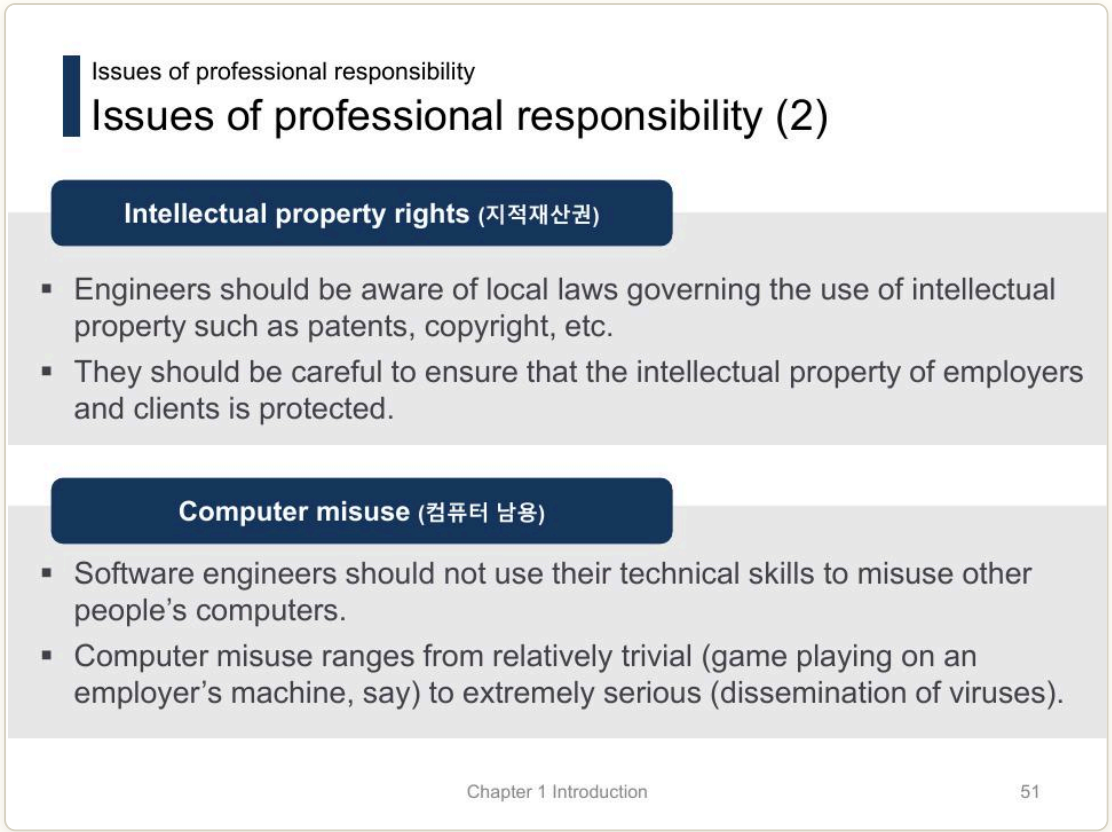


전문가 책임 이슈 ①

- **Confidentiality (비밀유지)** — 공식 비밀유지 계약을 했든 안 했든 고용주·고객의 비밀을 존중
- **Competence (능력)** — 자기 능력 수준을 속이지 말 것, 능력 밖의 일을 알면서 맡지 말 것

✍ 메모

— 넘어가도 되는 쪽

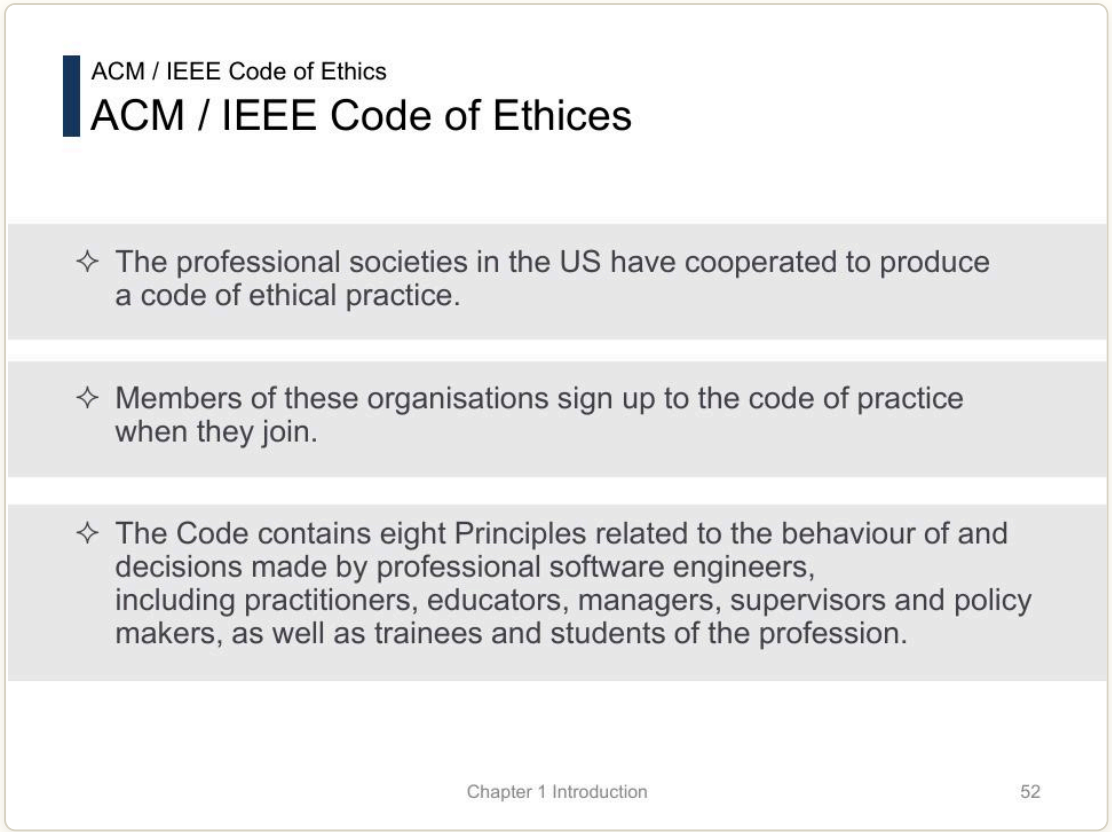


전문가 책임 이슈 ②

- **Intellectual property rights (지적재산권)** — 특허·저작권 등 관련 법을 알고, 고용주·고객의 지적재산을 보호
- **Computer misuse (컴퓨터 남용)** — 기술로 남의 컴퓨터를 오용하지 말 것
  - 사소한 것(회사 컴퓨터로 게임)부터 아주 심각한 것(바이러스 유포)까지

✍ 메모

★ 시험 책임 이슈 4개: Confidentiality · Competence · Intellectual property rights · Computer misuse



ACM/IEEE 윤리 강령

- 미국의 전문 학회들이 협력해 윤리 실천 강령을 만듦
- 회원은 가입할 때 강령에 동의
- 강령에는 전문 SW 엔지니어(실무자·교육자·관리자·감독자·정책 결정자, 수련생·학생 포함)의 행동과 결정에 관한 **8개 원칙**

✍️ 메모

★ 시험 원칙 개수 = 8

ACM / IEEE Code of Ethics

Rationale for the code of ethics

- *Computers have a central and growing role in commerce, industry, government, medicine, education, entertainment and society at large. Software engineers are those who contribute by direct participation or by teaching, to the analysis, specification, design, development, certification, maintenance and testing of software systems.*
- *Because of their roles in developing software systems, software engineers have significant opportunities to do good or cause harm, to enable others to do good or cause harm, or to influence others to do good or cause harm. To ensure, as much as possible, that their efforts will be used for good, software engineers must commit themselves to making software engineering a beneficial and respected profession.*

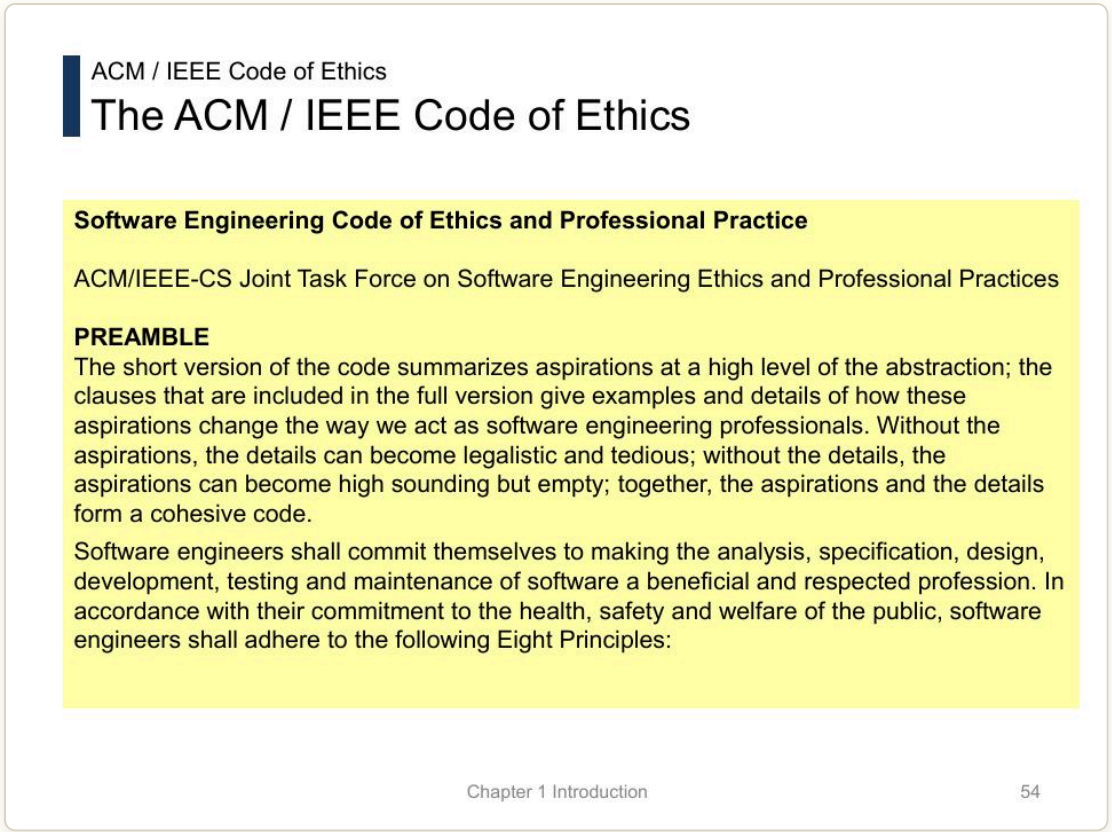
Chapter 1 Introduction 53

윤리 강령의 근거

- 컴퓨터는 상업·산업·정부·의료·교육·오락 등 사회 전반에서 중심적이고 커지는 역할
- SW 엔지니어는 분석·명세·설계·개발·인증·유지보수·테스트에 기여
- 그래서 좋은 일을 하거나 해를 끼칠 **큰 기회**가 있다
- 노력이 선하게 쓰이도록 SE를 유익하고 존중받는 직업으로 만드는 데 헌신해야

✍ 메모

— 넘어가도 되는 쪽

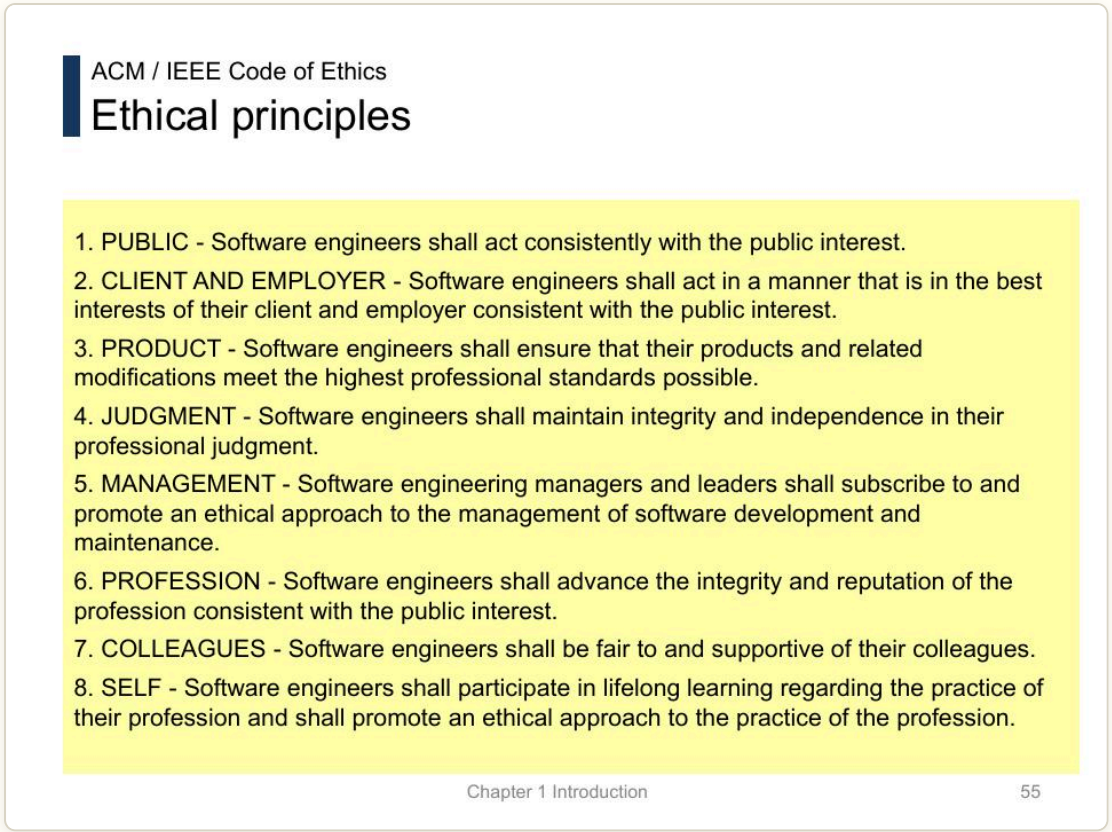


ACM/IEEE 윤리 강령 — 전문(前文)

- 짧은 판은 높은 수준의 열망을 요약, 긴 판은 그 열망이 행동을 어떻게 바꾸는지 예와 세부를 준다
  - 열망 없이 세부만 있으면 법률적이고 지루해지고, 세부 없이 열망만 있으면 공허해진다
- SW 엔지니어는 분석·명세·설계·개발·테스트·유지보수를 유익하고 존중받는 직업으로 만드는 데 헌신
- 공공의 건강·안전·복지에 대한 약속에 따라 **8개 원칙**을 지킨다

✍️ 메모

— 넘어가도 되는 쪽



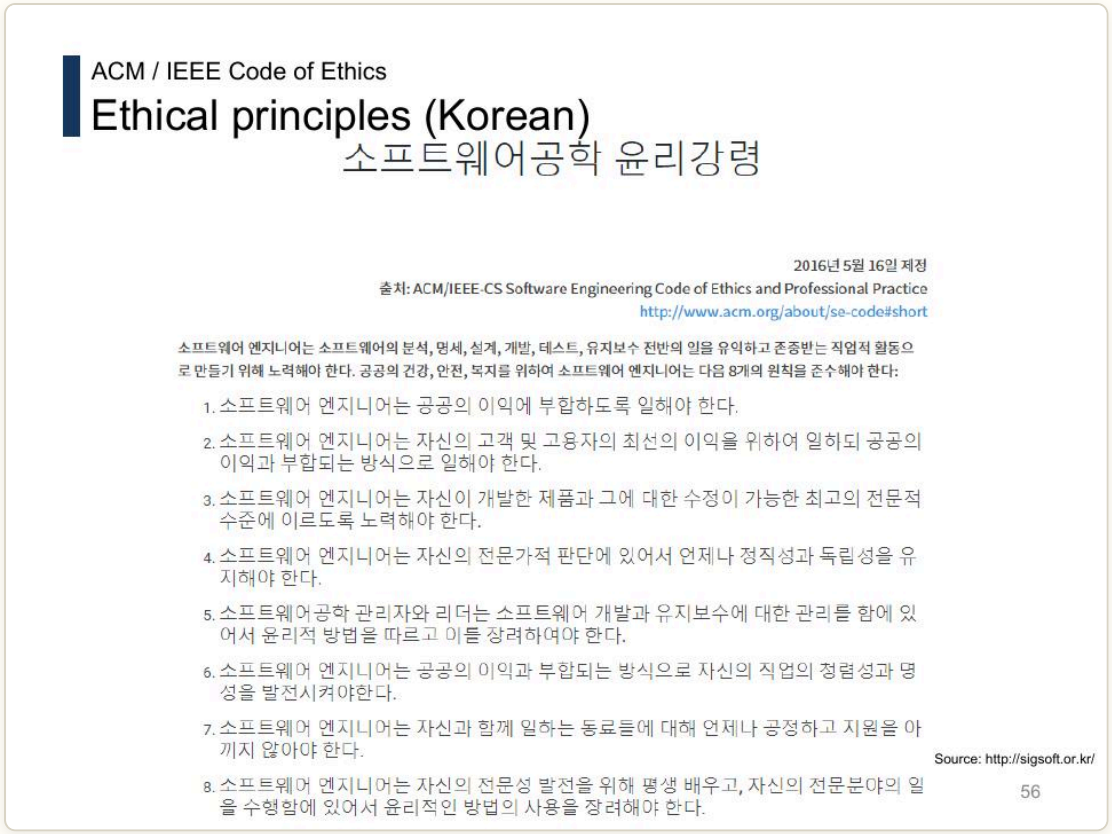
윤리 원칙 8가지

- 1. **PUBLIC** — 공공의 이익에 맞게 행동
- 2. **CLIENT AND EMPLOYER** — 공익에 맞는 한 고객·고용주의 최선의 이익
- 3. **PRODUCT** — 제품과 수정이 가능한 최고의 전문 표준을 충족
- 4. **JUDGMENT** — 전문적 판단의 진실성과 독립성 유지
- 5. **MANAGEMENT** — 관리자·리더는 윤리적 관리 방식을 따르고 장려
- 6. **PROFESSION** — 공익에 맞게 직업의 진실성과 평판을 높임
- 7. **COLLEAGUES** — 동료에게 공정하고 협조적
- 8. **SELF** — 평생 학습, 윤리적 실천을 장려

✍️ 메모

★ 시험 8원칙 키워드 순서: Public · Client&Employer · Product · Judgment · Management · Profession · Colleagues · Self

💡 이해 외우기: 공·고·제·판·관·직·동·자



윤리 원칙 (한국어판)

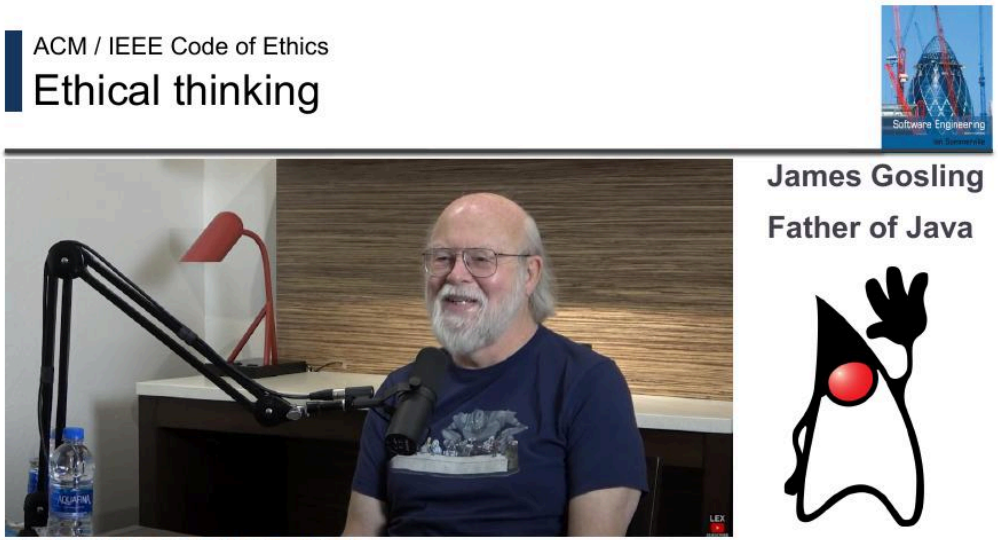
- 한국소프트웨어공학회(sigsoft.or.kr)의 한국어 번역 (그림)

✍ 메모

— 넘어가도 되는 쪽

ACM / IEEE Code of Ethics

Ethical thinking



James Gosling

Father of Java

https://www.youtube.com/watch?v=IT__Nrr3PNI

" I starting to **think about ethical choices in my life**.

I'm a big science fiction fan.

I got to thinking about just about technical decision (that) I make.

In terms of "Are you building Blade Runner or Start Track" **which future would you rather live in?** "

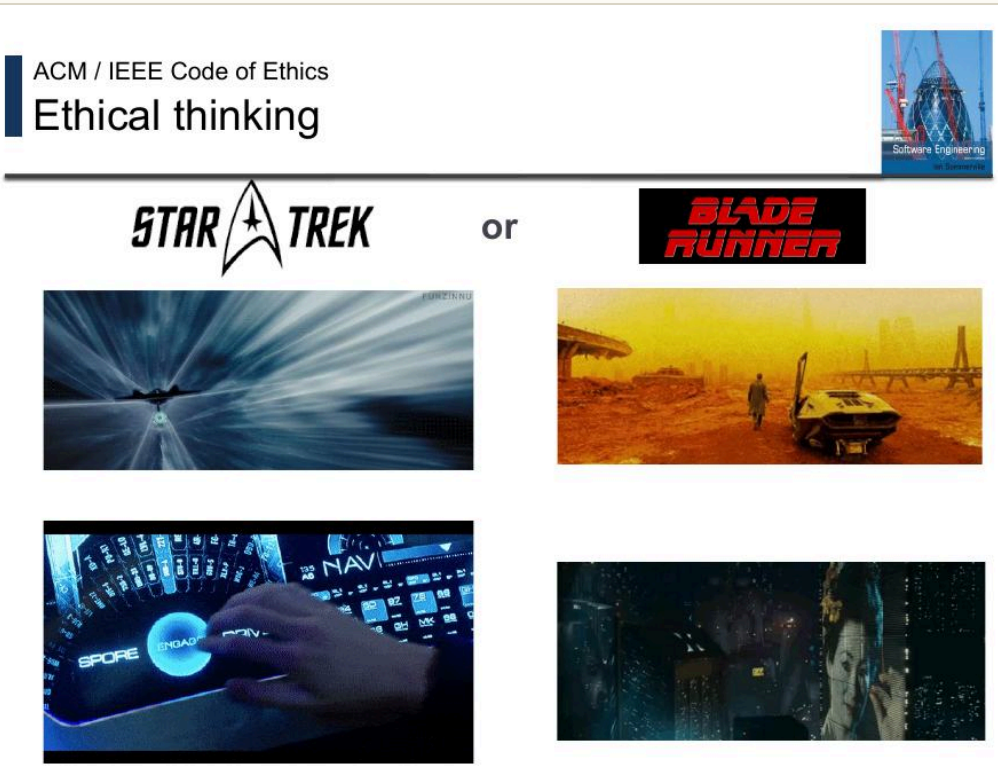
57

윤리적 사고 — James Gosling

- Java의 아버지 James Gosling
- "내 삶의 윤리적 선택을 생각하기 시작했다. SF 팬으로서 기술적 결정마다 생각한다 — 블레이드 러너를 만들고 있나, 스타트렉을 만들고 있나? 어느 미래에 살고 싶은가?"

✍ 메모

✏ 예·풀이 Java 창시자 = James Gosling (오답 보기로 Kent Beck, Stroustrup 이 나올 수 있음)

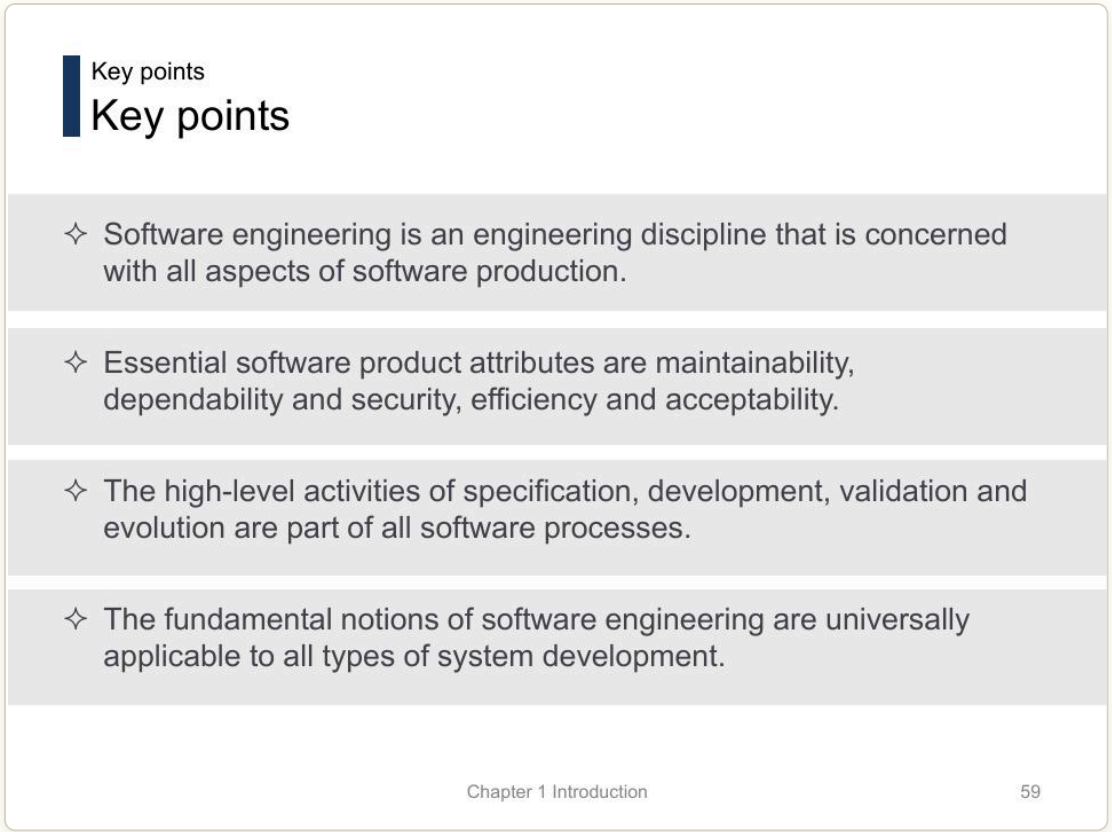


윤리적 사고 (그림)

- 블레이드 러너(디스토피아) or 스타트렉(희망적 미래)?

✍ 메모

— 넘어가도 되는 쪽

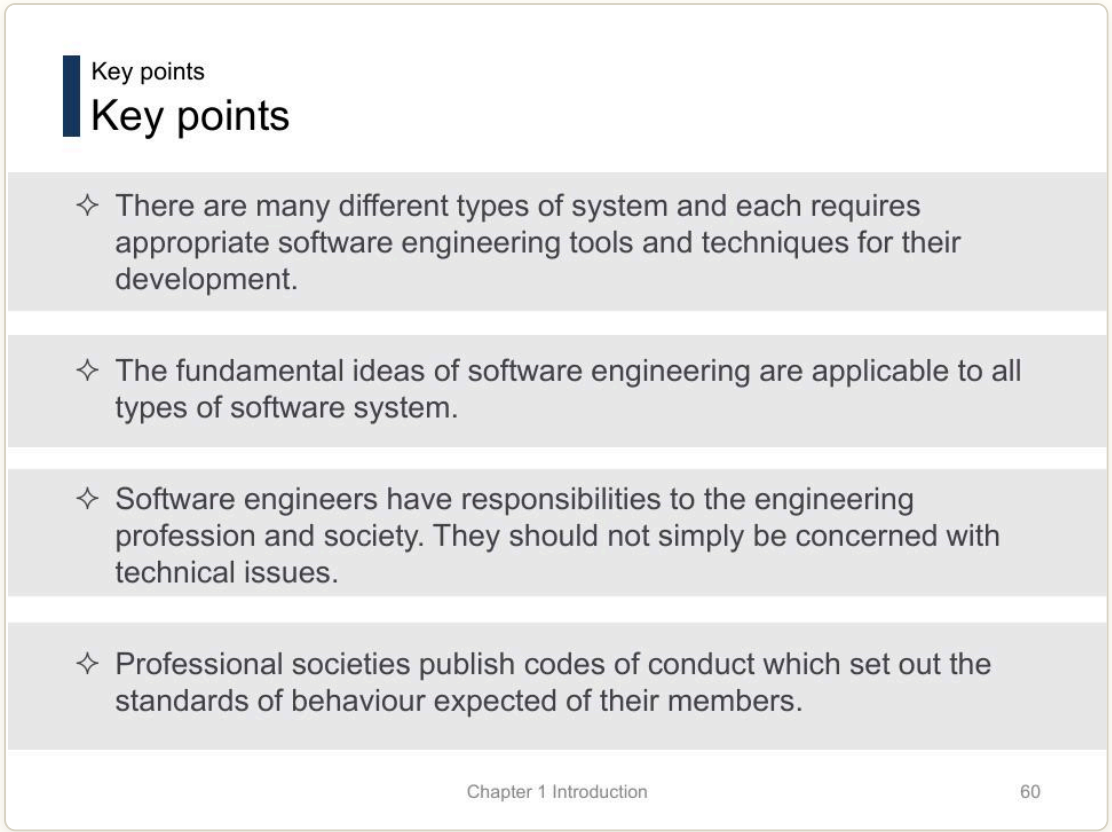


핵심 정리 ①

- SE는 SW 생산의 **모든 측면**을 다루는 공학 분야
- 필수 제품 속성: **유지보수성, 확실성과 보안성, 효율성, 수용성**
- 명세·개발·검증·진화라는 상위 활동은 **모든 SW 프로세스**의 일부
- SE의 기본 개념은 **모든 종류**의 시스템 개발에 보편적으로 적용

✍️ 메모

★ 시험 **Key points = 시험 문장 그대로 나오기 좋음**

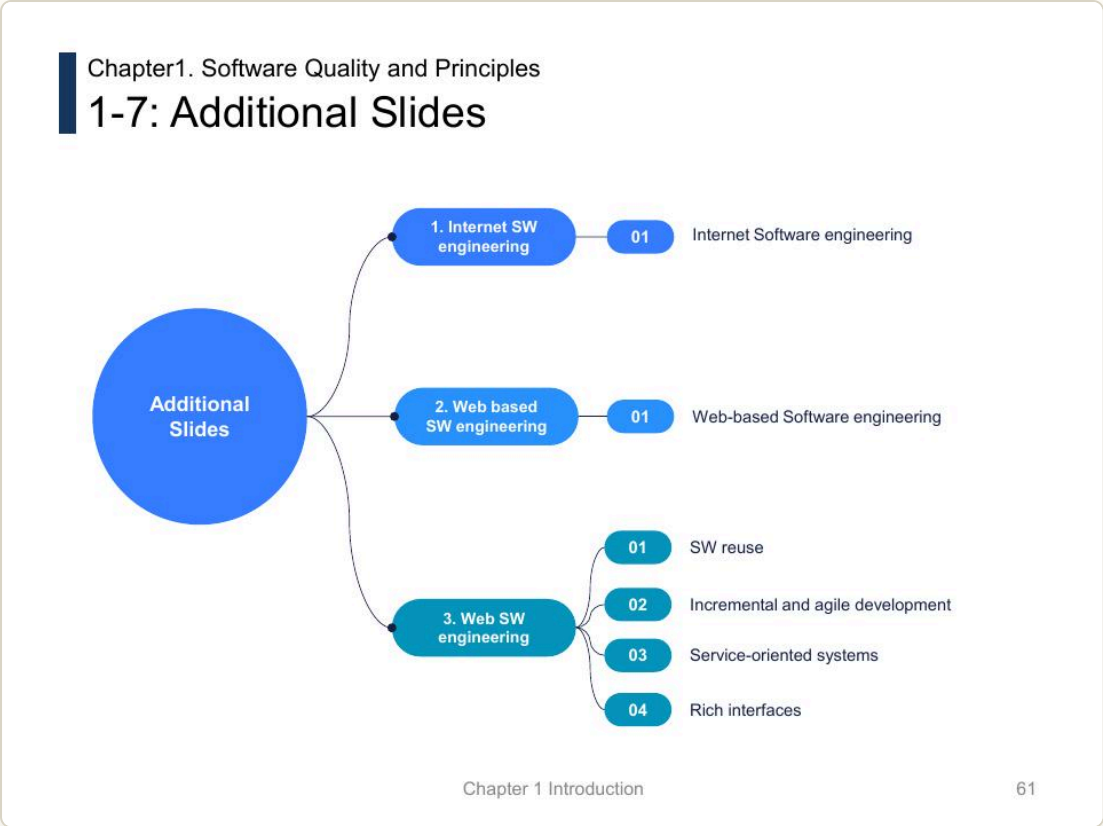


핵심 정리 ②

- 시스템 종류가 많고, 각각 알맞은 SE 도구·기법이 필요
- SE의 기본 아이디어는 모든 종류의 SW 시스템에 적용 가능
- SW 엔지니어는 직업과 사회에 책임이 있다 — 기술 문제만 신경 쓰면 안 됨
- 전문 학회는 회원에게 기대하는 행동 기준(강령)을 발표

✍️ 메모

— 넘어가도 되는 쪽

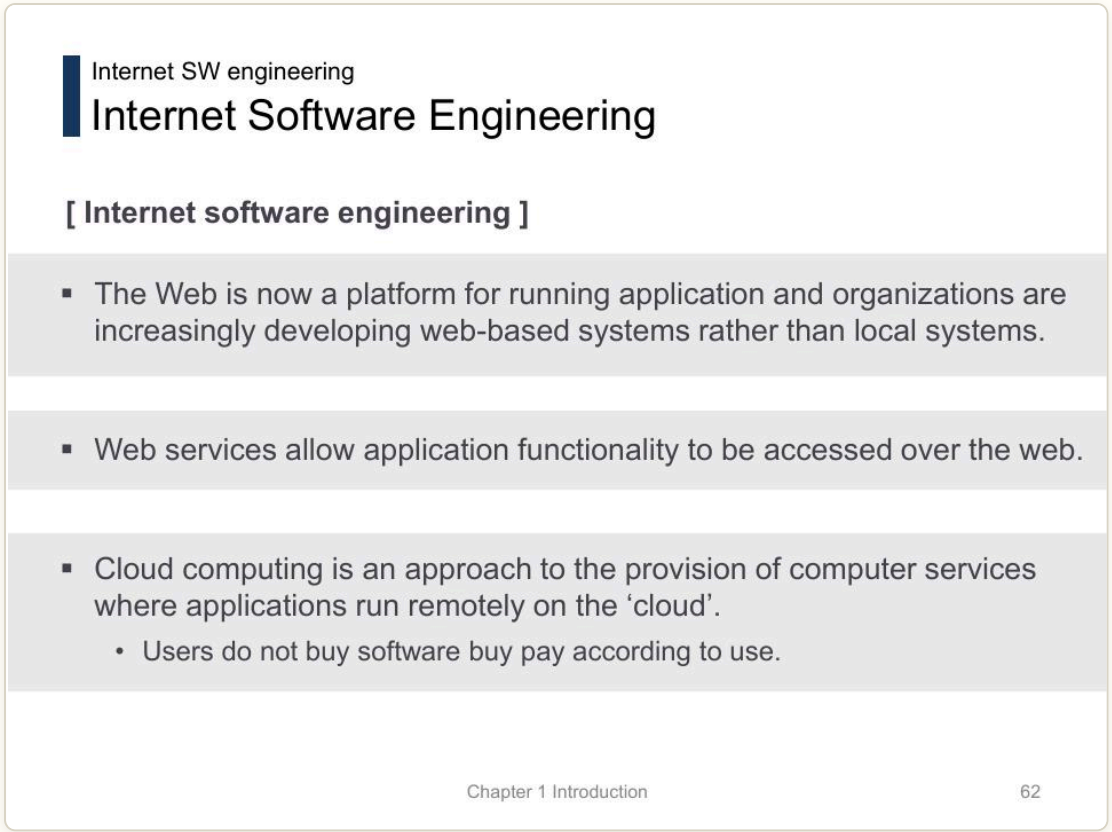


1-7: 추가 슬라이드

- 인터넷 SE
- 웹 기반 SE
- 웹 SE — SW 재사용, 점증·애자일 개발, 서비스 지향 시스템, 풍부한 인터페이스

✍️ 메모

💡 이해 절 표지 — 아래 쪽들의 지도

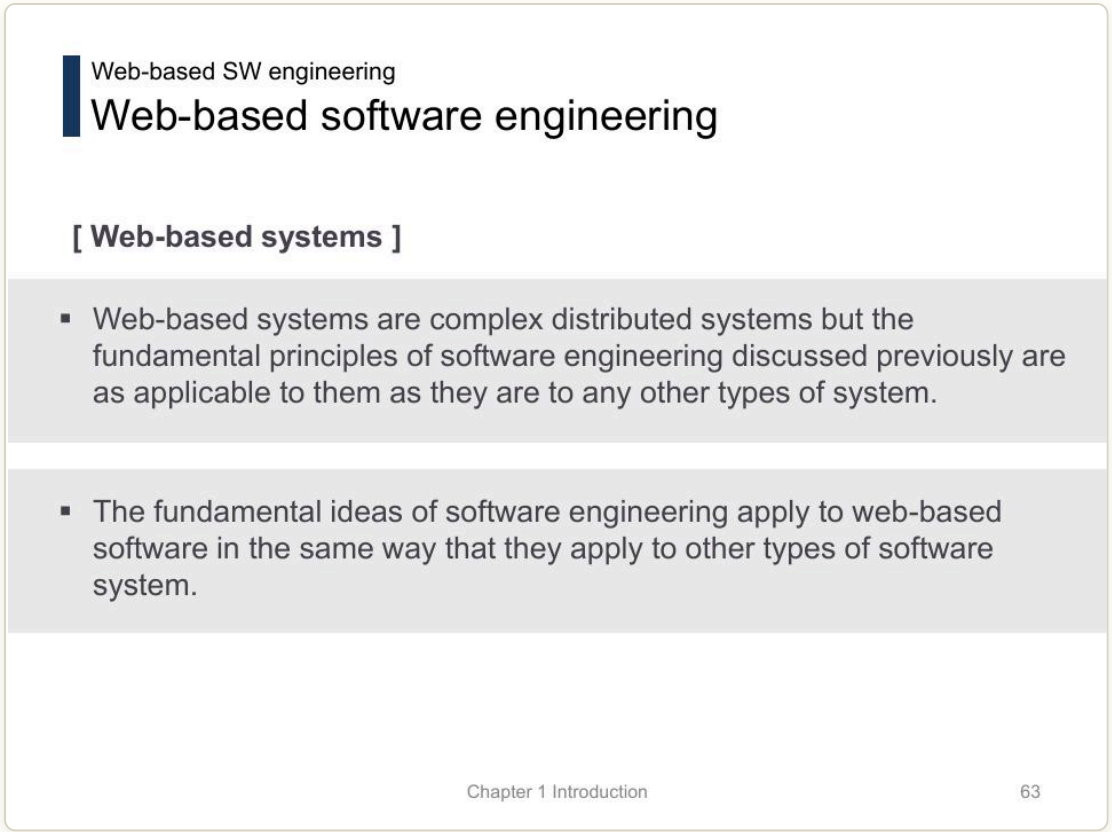


인터넷 SE

- 웹이 이제 응용을 실행하는 **플랫폼** — 조직은 로컬 시스템보다 웹 기반 시스템을 점점 더 개발
- **웹 서비스**로 응용 기능을 웹에서 접근
- **클라우드 컴퓨팅**: 응용이 원격 '클라우드'에서 실행
  - 사용자는 SW를 사지 않고 **쓴 만큼** 낸다

✍ 메모

— 넘어가도 되는 쪽

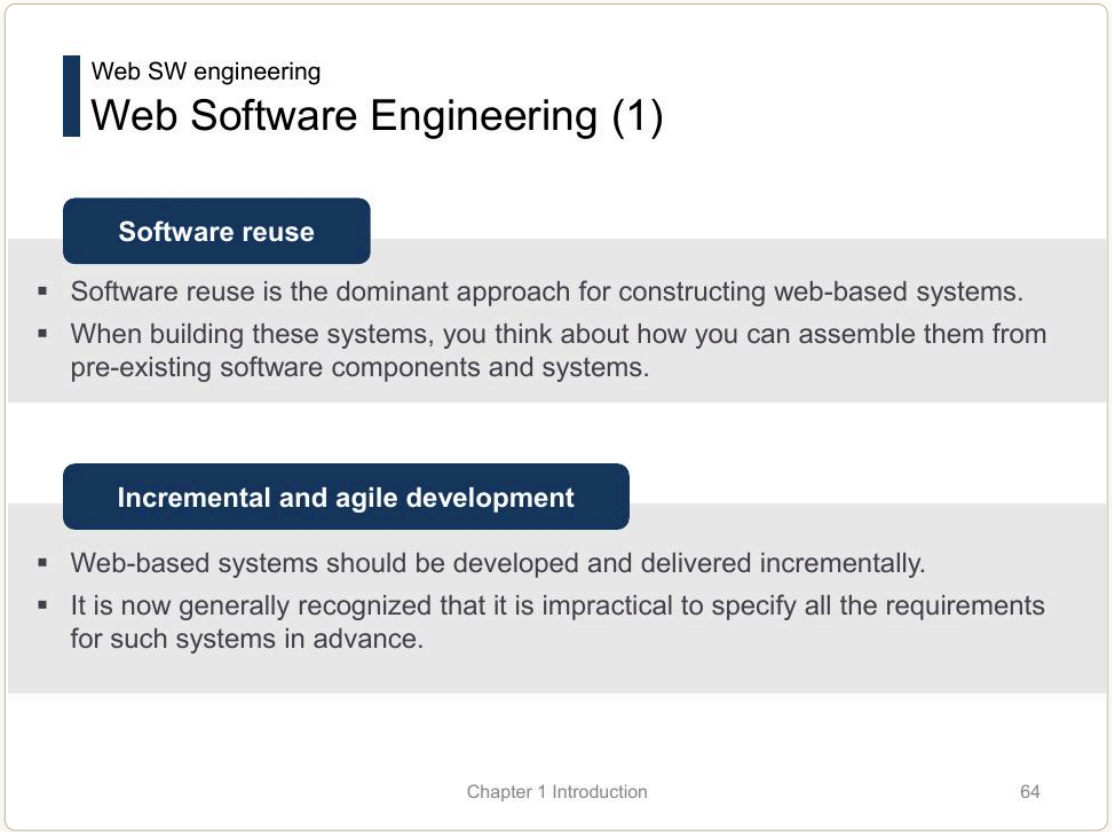


웹 기반 SE

- 웹 기반 시스템은 복잡한 분산 시스템이지만, 앞서 본 SE 기본 원리가 똑같이 적용된다
- SE의 기본 아이디어는 다른 SW처럼 웹 SW에도 적용

✍ 메모

— 넘어가도 되는 쪽

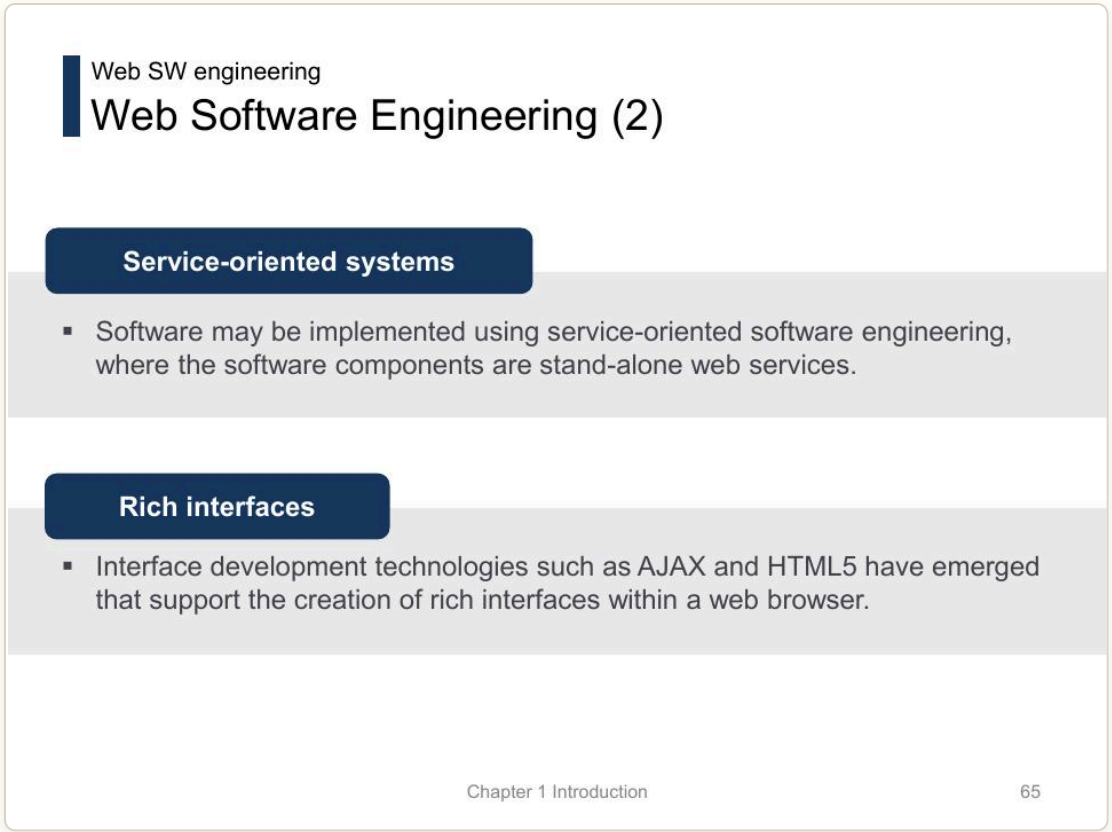


웹 SE ①

- **Software reuse** — 웹 기반 시스템을 만드는 **지배적 방법**
  - 기존 SW 구성 요소·시스템을 어떻게 조립할지 생각
- **Incremental and agile development** — 웹 기반 시스템은 **점증적으로** 개발·인도
  - 모든 요구사항을 미리 명세하는 것은 비현실적이라는 게 일반적 인식

✍ 메모

— 넘어가도 되는 쪽



웹 SE ②

- **Service-oriented systems** — 구성 요소가 독립 웹 서비스인 서비스 지향 SE로 구현
- **Rich interfaces** — AJAX, HTML5 같은 기술로 웹 브라우저 안에서 풍부한 인터페이스

✍ 메모

★ 시험 웹 SE 4개: Reuse · Incremental/agile · Service-oriented · Rich interfaces